CHAPTER OUTLINE


4.1 **Field and Record Organization**
- 4.1.1 A Stream File
- 4.1.2 Field Structures
- 4.1.3 Reading a Stream of Fields
- 4.1.4 Record Structures
- 4.1.5 A Record Structure That Uses a Length Indicator
- 4.1.6 Mixing Numbers and Characters: Use of a File Dump

4.2 **Record Access**
- 4.2.1 Record Keys
- 4.2.2 A Sequential Search
- 4.2.3 UNIX Tools for Sequential Processing
- 4.2.4 Direct Access

4.3 **More about Record Structures**
- 4.3.1 Choosing a Record Structure and Record Length
- 4.3.2 Header Records

4.4 **File Access and File Organization**

4.5 **Beyond Record Structures**
- 4.5.1 Abstract Data Models
- 4.5.2 More Complex Headers
- 4.5.3 Metadata
- 4.5.4 Color Raster Images
- 4.5.5 Mixing Object Types in One File
- 4.5.6 Object-oriented File Access
- 4.5.7 Extensibility

4.6 **Portability and Standardization**
- 4.6.1 Factors Affecting Portability
- 4.6.2 Achieving Portability


## 4.1 Field and Record Organization

When we build file structures we are imposing order on data. In this chapter we investigate the many forms that this ordering can take. We begin by looking at the base case: a file organized as a stream of bytes.

### 4.1.1 A Stream File

Suppose the file we are building contains name and address information. A program to accept names and addresses from the keyboard, writing them out as a stream of consecutive bytes to a file with the logical name OUTPUT, is described in the pseudocode shown in Fig. 4.1.

Implementations of this program in both C and Pascal, called *writstrm.c* and *writstrm.pas*, are provided in the C and Pascal Programs sections at the end of this chapter. You should type in this program, working in either C or Pascal, compile it, and run it. We use it as the basis for a number of experiments, and you can get a better feel for the differences between



```
PROGRAM: writstrm

    get output file name and open it with the logical name OUTPUT

    get LAST name as input
    while ( LAST name has a length > 0)
        get FIRST name, ADDRESS, CITY, STATE and ZIP as input

        write LAST    to the file OUTPUT
        write FIRST   to the file OUTPUT
        write ADDRESS to the file OUTPUT
        write CITY    to the file OUTPUT
        write STATE   to the file OUTPUT
        write ZIP     to the file OUTPUT

        get LAST name as input
    endwhile

    close OUTPUT

end PROGRAM
```

**FIGURE 4.1** Program to write out a name and address file as a stream of bytes.

the file structures we are discussing if you perform the experiments yourself.

The following names and addresses are used as input to the program:

```
John Ames                Alan Mason
123 Maple                90 Eastgate
Stillwater, OK 74075     Ada, OK 74820
```

When we list the output file on our terminal screen, here is what we see:

```
AmesJohn123 MapleStillwaterOK74075MasonAlan90 EastgateAdaOK74820
```

The program writes the information out to the file precisely as specified: as a stream of bytes containing no added information. But in meeting our specifications, the program creates a kind of "reverse Humpty-Dumpty" problem. Once we put all that information together as a single byte stream, there is no way to get it apart again.

We have lost the integrity of the fundamental organizational units of our input data; these fundamental units are not the individual characters, but meaningful aggregates of characters, such as "John Ames" or "123 Maple."

When we are working with files, we call these fundamental aggregates *fields*. A field is *the smallest logically meaningful unit of information in a file.*[†]

A field is a logical notion; it is a *conceptual tool*. A field does not necessarily exist in any physical sense, yet it is important to the file's structure. When we write out our name and address information as a stream of undifferentiated bytes, we lose track of the fields that make the information meaningful. We need to organize the file in some way that lets us keep the information divided into fields.

### 4.1.2 Field Structures

There are many ways of adding structure to files to maintain the identity of fields. Four of the most common methods are

- Force the fields into a predictable length.
- Begin each field with a length indicator.
- Place a *delimiter* at the end of each field to separate it from the next field.
- Use a "keyword = value" expression to identify each field and its contents.

**Method 1: Fix the Length of Fields** The fields in our sample file vary in their length. If we force the fields into predictable lengths, then we can pull them back out of the file simply by counting our way to the end of the field. We can define a structure in C or a record in Pascal to hold these fixed-length fields, as shown in Fig. 4.2.

Using this kind of fixed-field length structure changes our output so it looks like that shown in Fig. 4.3(a). Simple arithmetic is sufficient to let us recover the data in terms of the original fields.

One obvious disadvantage of this approach is that adding all the padding required to bring the fields up to a fixed length makes the file much larger. Rather than using 4 bytes to store the last name *Ames*, we use 10. We can also encounter problems with data that is too long to fit into the allocated amount of space. We could solve this second problem by fixing all the fields at lengths that are large enough to cover all cases, but this would just make the first problem of wasted space in the file even worse.

[†]Readers should not confuse the term *field* and *record* with the meanings given to them by some programming languages, including Pascal. In Pascal, a record is an aggregate data structure that can contain members of different types, where each member is referred to as a field. As we shall see, there is often a direct correspondence between these definitions of the terms and the fields and records that are used in files. However, the terms *field* and *record* as we use them have much more general meanings than they do in Pascal.



**In C:**

```
struct {
    char last[10];
    char first[10];
    char address[15];
    char city[15];
    char state[2];
    char zip[9];
} set_of_fields;
```

**In Pascal:**

```
TYPE
  set_of_fields = RECORD
      last    : packed array [1..10] of char;
      first   : packed array [1..10] of char;
      address : packed array [1..15] of char;
      city    : packed array [1..15] of char;
      state   : packed array [1..2]  of char;
      zip     : packed array [1..9]  of char;
  END;
```

**FIGURE 4.2** Fixed-length fields.

Because of these difficulties, the fixed-field approach to structuring data is often inappropriate for data that inherently contain a large amount of variability in the length of fields, such as names and addresses. But there are kinds of data for which fixed-length fields are highly appropriate. If every field is already fixed in length, or if there is very little variation in field lengths, using a file structure consisting of a continuous stream of bytes organized into fixed-length fields is often a very good solution.

**Method 2: Begin Each Field with a Length Indicator** Another way to make it possible to count to the end of a field involves storing the field length just ahead of the field, as illustrated in Fig. 4.3(b). If the fields are not too long (length less than 256 bytes), it is possible to store the length in a single byte at the start of each field.

**Method 3: Separate the Fields with Delimiters** We can also preserve the identity of fields by separating them with delimiters. All we need to do is choose some special character or sequence of characters that will not appear within a field and then *insert* that delimiter into the file after writing each field.

The choice of a delimiter character can be very important since it must be a character that does not get in the way of processing. In many instances *white-space characters* (blank, new line, tab) make excellent delimiters because they provide a clean separation between fields when we list them on the console. Also, most programming languages include I/O statements that, by default, assume that fields are separated by white space.

Unfortunately, white space would be a poor choice for our file since blanks often occur as legitimate characters within an address field.

```
Ames       John    123 Maple    Stillwater  OK74075377-1808
Mason      Alan    90 Eastgate  Ada         OK74820
```

(a) Field lengths fixed. Place blanks in the spaces where the phone number would go.

```
Ames|John|123 Maple|Stillwater|OK|74075|377-1808|
Mason|Alan|90 Eastgate|Ada|OK|74820||
```

(b) Delimiters are used to indicate the end of a field. Place the delimiter for the "empty" field immediately after the delimiter for the previous field.

```
Ames|_|Stillwater|OK|74075|377-1808|#Mason|_ 90Eastgate|Ada|OK|74820|#...
```

(c) Place the field for business phone at the end of the record. If the end-of-record mark is encountered, assume that the field is missing.

```
SURNAME=Ames|FIRSTNAME=John|STREET=123 Maple|...|ZIP=74075|PHONE=377-1808|#_
```

(d) Use a keyword to identify each field. If the keyword is missing, the corresponding field is assumed to be missing.

**FIGURE 4.3** Four methods for organizing fields within records to account for possible missing fields. In the examples, the second record is missing the phone number.

Therefore, instead of white space we use the vertical bar character as our delimiter, so our file appears as in Fig. 4.3(c). Readers should modify the original stream-of-bytes programs, *writstrm.c* and *writstrm.pas* (found in the C and Pascal Programs sections at the end of this chapter), changing them so they place a delimiter after each field. We use this delimited field format in the next few sample programs.

**Method 4: Use a "Keyword = Value" Expression to Identify Fields**

This option, illustrated in Fig. 4.2(d), has an advantage that the others do not: It is the first structure in which a field provides information about itself. Such *self-describing* structures can be very useful tools for organizing files



in many applications. It is easy to tell what fields are contained in a file, even if we don't know ahead of time what fields the file is supposed to contain. It is also a good format for dealing with missing fields. If a field is missing, this format makes it obvious, because the keyword is simply not there.

You may have noticed in Fig. 4.3(d) that this format is used in combination with another format, a delimiter to separate fields. While this may not always be necessary, in this case it is helpful because it shows the division between each value and the keyword for the following field.

Unfortunately, for the address file this format also wastes a lot of space. Fifty percent or more of the file's space could be taken up by the keywords. But there are applications in which this format does not demand so much overhead. We discuss some of these applications in section 4.5.

### 4.1.3 Reading a Stream of Fields

Given modified versions of *writstrm.c* and *writstrm.pas* that use delimiters to separate fields, we can write a program called *readstrm* that reads the stream of bytes back in, breaking the stream into fields. It is convenient to conceive of the program on two levels, as shown in the pseudocode description provided in Fig. 4.4. The outer level of the program opens the file and then calls the function *readfield( )* until *readfield( )* returns a field length of zero, indicating that there are no more fields to read. The *readfield( )* function, in turn, works through the file, character by character, collecting characters into a field until the function encounters a delimiter or the end of the file. The function returns a count of the characters that are found in the field. Implementations of *readstrm* in both C and Pascal are included with the programs at the end of this chapter.

When this program is run using our delimited-field version of the file containing data for John Ames and Alan Mason, the output looks like this:

```
Field #  1: Ames
Field #  2: John
Field #  3: 123 Maple
Field #  4: Stillwater
Field #  5: OK
Field #  6: 74075
Field #  7: Mason
Field #  8: Alan
Field #  9: 90 Eastgate
Field # 10: Ada
Field # 11: OK
Field # 12: 74820
```

```
Define Constant: DELIMITER = '|'

PROGRAM: readstrm

    get input file name and open as INPUT
    initialize FIELD_COUNT

    FIELD_LENGTH := readfield (INPUT, FIELD_CONTENT)
    while ( FIELD_LENGTH > 0 )

        increment the FIELD_COUNT
        write FIELD_COUNT and FIELD_CONTENT to the screen
        FIELD_LENGTH := readfield (INPUT, FIELD_CONTENT)

    endwhile

    close INPUT
end PROGRAM


FUNCTION: readfield (INPUT, FIELD_CONTENT)

    initialize I
    initialize CH

    while (not EOF (INPUT) and CH does not equal DELIMITER)

        read a character from INPUT into CH
        increment I
        FIELD_CONTENT [I] := CH

    endwhile
    return (length of field that was read)

end FUNCTION
```

**FIGURE 4.4** Program to read fields from a file and display them on the screen.

Clearly, we now preserve the notion of a field as we store and retrieve these data. But something is still missing. We do not really think of this file as a stream of fields. In fact, the fields need to be grouped into sets. The first six fields are a set associated with someone named John Ames. The next six are a set of fields associated with Alan Mason. We call these sets of fields *records*.



### 4.1.4 Record Structures

A *record* can be defined as *a set of fields that belong together when the file is viewed in terms of a higher level of organization*. Like the notion of a field, a record is another conceptual tool. It is another level of organization that we impose on the data to preserve meaning. Records do not necessarily exist in the file in any physical sense, yet they are an important logical notion included in the file's structure.

Here are some of the most often used methods for organizing a file into records:

- Require that the records be a predictable number of bytes in length.
- Require that the records be a predictable number of fields in length.
- Begin each record with a length indicator consisting of a count of the number of bytes that the record contains.
- Use a second file to keep track of the beginning byte address for each record.
- Place a delimiter at the end of each record to separate it from the next record.

**Method 1: Make Records a Predictable Number of Bytes (Fixed-length Records)** A *fixed-length record file* is one in which each record contains the same number of bytes. This method of recognizing records is analogous to the first method we discussed for making fields recognizable. As we will see in the chapters that follow, fixed-length record structures are among the most commonly used methods for organizing files.

The C structure *set_of_fields* (or the Pascal RECORD of the same name) that we define in our discussion of fixed-length fields is actually an example of a fixed-length *record* as well as an example of fixed-length fields. We have a fixed number of fields, each with a predetermined length, which combine to make a fixed-length record. This kind of field and record structure is illustrated in Fig. 4.5(a).

It is important to realize, however, that fixing the number of bytes in a record does not imply that the sizes or number of fields in the record must be fixed. Fixed-length records are frequently used as containers to hold variable numbers of variable length fields. It is also possible to mix fixed- and variable-length fields within a record. Figure 4.5(b) illustrates how variable-length fields might be placed in a fixed-length record.

**Method 2: Make Records a Predictable Number of Fields** Rather than specifying that each record in a file contain some fixed number of bytes, we can specify that it will contain a fixed number of fields. This is a good way to organize the records in the name and address file we have been

| | | | | |
|---|---|---|---|---|
| Ames | John | 123 Maple | Stillwater | OK74075 |
| Mason | Alan | 90 Eastgate | Ada | OK74820 |

(a)

Ames¦John¦123 Maple¦Stillwater¦OK¦74075¦ ← Unused space →

Mason¦Alan¦90 Eastgate¦Ada¦OK¦74820¦ ← Unused space →

(b)

Ames¦John¦123 Maple¦Stillwater¦OK¦74075¦Mason¦Alan¦90 Eastgate¦Ada¦OK . . .

(c)

**FIGURE 4.5** Three ways of making the lengths of records constant and predictable. (a) Counting bytes: fixed-length records with fixed-length fields. (b) Counting bytes: fixed-length records with variable-length fields. (c) Counting fields: six fields per record.

looking at. The *writstrm* program asks for six pieces of information for every person, so there are six contiguous fields in the file for each record (Fig. 4.5c). We could modify *readstrm* to recognize fields simply by counting the fields *modulo* six, outputting record boundary information to the screen every time the count starts over.

**Method 3: Begin Each Record with a Length Indicator** We can communicate the length of records by beginning each record with a field containing an integer that indicates how many bytes there are in the rest of the record (Fig. 4.6a). This is a commonly used method for handling variable-length records. We look at it more closely in the next section.

**Method 4: Use an Index to Keep Track of Addresses** We can use an *index* to keep a byte offset for each record in the original file. The byte offsets allow us to find the beginning of each successive record and also let us compute the length of each record. We look up the position of a record in the index and then seek to the record in the data file. Figure 4.6(b) illustrates this two-file mechanism.

**Method 5: Place a Delimiter at the End of Each Record** This option, at a record level, is exactly analogous to the solution we used to keep the



*fields* distinct in the sample program we developed. As with fields, the delimiter character must not get in the way of processing. Because we often want to read files directly at our console, a common choice of a record delimiter for files that contain readable text is the end-of-line character (carriage return/new-line pair or, on UNIX systems, just a new-line character—'\n'). In Fig 4.6(c) we use a '#' character as the record delimiter.

### 4.1.5 A Record Structure That Uses a Length Indicator

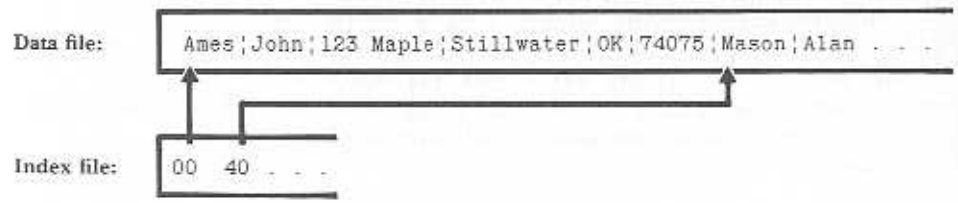
Not one of these approaches to preserving the idea of a *record* in a file is appropriate for all situations. Selection of a method for record organization depends on the nature of the data and on what you need to do with it. We begin by looking at a record structure that uses a record-length field at the beginning of the record. This approach lets us preserve the *variability* in the length of records that is inherent in our initial stream file.

**Writing the Variable-length Records to the File** We call the program that builds this new, variable-length record structure *writrec*. The set of programs at the end of this chapter contains versions of this program in C and Pascal. Implementing this program is partially a matter of building on

**FIGURE 4.6** Record structures for variable-length records. (a) Beginning each record with a length indicator. (b) Using an index file to keep track of record addresses. (c) Placing the delimiter '#' at the end of each record.

40Ames¦John¦123 Maple¦Stillwater¦OK¦74075¦36Mason¦Alan¦90 Eastgate . . .

(a)




(b)

Ames¦John¦123 Maple¦Stillwater¦OK¦74075¦#Mason¦Alan¦90 Eastgate¦Ada¦OK . . .

(c)

the *writstrm* program that we created earlier in this chapter, but also involves addressing some new problems:

- If we want to put a length indicator at the *beginning* of every record (before any other fields), we must know the sum of the lengths of the fields in each record before we can begin writing the record to the file. We need to accumulate the entire contents of a record in a *buffer* before writing it out.
- In what form should we write the record-length field to the file? As a binary integer? As a series of ASCII characters?

The concept of buffering is one we run into again and again as we work with files. In the case of *writrec*, the buffer can simply be a character array into which we place the fields and field delimiters as we collect them. Resetting the buffer length to zero and adding information to the buffer can be handled using the loop logic provided in Fig. 4.7.

**Representing the Record Length** The question of how to represent the record length is a little more difficult. One option would be to write the length in the form of a two-byte binary integer before each record. This is a natural solution in C, since it does not require us to go to the trouble of converting the record length into character form. Furthermore, we can represent much bigger numbers with an integer than we can with the same number of ASCII bytes (e.g., 32,767 versus 99). It is also conceptually

**FIGURE 4.7** Main program logic for *writrec*.

```
get LAST name as input
while ( LAST name has a length > 0 )
    set length of string in BUFFER to zero
    concatenate: BUFFER + LAST name + DELIMITER

    while ( input fields exist for record )
        get the FIELD
        concatenate: BUFFER + FIELD + DELIMITER
    endwhile

    write length of string in BUFFER to the file
    write the string in BUFFER to the file

    get LAST name as input
endwhile
```



interesting, since it illustrates the use of a fixed-length, binary field in combination with variable-length character fields.

Although we could use this same solution for a Pascal implementation, we might choose, instead, to account for some important differences between C and Pascal:

- Unlike C, Pascal automatically converts binary integers into character representations of those integers if we are writing to a text file. Consequently, it is no trouble at all to convert the record length into a character form: It happens automatically.
- In Pascal, a file is defined as a sequence of elements of a single type. Since we have a file of variable-length strings of characters, the natural type for the file is that of a character.

In short, the easiest thing to do in C is to store the integers in the file as fixed-length, two-byte fields containing integers. In Pascal it is easier to make use of the automatic conversion of integers into characters for text files. File structure design is always an exercise in flexibility. Neither of these approaches is correct; good design consists of choosing the approach that is most *appropriate* for a given language and computing environment. In the programs included at the end of this chapter, we have implemented our record structure both ways, using integer-length fields in C and character representations in Pascal. The output from the Pascal implementation is shown in Fig. 4.8. Each record now has a record-length field preceding the data fields. This field is delimited by a blank. For example, the first record (for John Ames) contains 40 characters, counting from the first 'A' in "Ames" to the final delimiter after "74075," so the characters '4' and '0' are placed before the record, followed by a blank.

Since the C version of *writrec* uses binary integers for the record length, we cannot simply print it to a console screen. We need a way to interpret the noncharacter portion of the file. For this, we introduce in the next section the file dump, a valuable tool for viewing the contents of files. But first, let's look at a program that will read in any file that is written by *writrec*.

**Reading the Variable-length Records from the File** Given our file structure of variable-length records preceded by record-length fields, it is

**FIGURE 4.8** Records preceded by record-length fields in character form.

```
40 Ames¦John¦123 Maple¦Stillwater¦OK¦74075¦36 Mason¦Alan¦90
Eastgate¦Ada¦OK¦74820¦
```

```
PROGRAM: readrec

    open input file as INP_FILE
    initialize SCAN_POS to 0
    RECORD_LENGTH := get_rec(INP_FILE, BUFFER)
    while (RECORD_LENGTH > 0)
        SCAN_POS := get_fld(FIELD,BUFFER,SCAN_POS,RECORD_LENGTH)
        while (SCAN_POS > 0)
            print FIELD on the SCREEN
            SCAN_POS := get_fld(FIELD,BUFFER,SCAN_POS,RECORD_LENGTH)
        endwhile

        RECORD_LENGTH := get_rec(INP_FILE, BUFFER)
    endwhile
end PROGRAM


FUNCTION: get_rec(INP_FILE, BUFFER)

    if EOF (INP_FILE) then return 0

    read the RECORD_LENGTH
    read the record contents into the BUFFER
    return the RECORD_LENGTH

end FUNCTION


FUNCTION: get_fld(FIELD,BUFFER,SCAN_POS,RECORD_LENGTH)

    if SCAN_POS == RECORD_LENGTH then return 0

    get a character CH at the SCAN_POS in the BUFFER
    while (SCAN_POS < RECORD_LENGTH and CH is not a DELIMITER)
        place CH into the FIELD
        increment the SCAN_POS
        get a character CH at the SCAN_POS in the BUFFER
    endwhile


    return the SCAN_POS

end FUNCTION
```

**FIGURE 4.9** Main program logic for *readrec*, along with functions *get_rec( )* and *get_fld( )*.



easy to write a program that reads through the file, record by record, displaying the fields from each of the records on the screen. The program logic is shown in Fig. 4.9. The main program calls the function *get_rec( )* that reads records into a buffer; this call continues until *get_rec( )* returns a value of 0. Once *get_rec( )* places a record's contents into a buffer, the buffer is passed to the function *get_fld( )*. The call to *get_fld( )* includes a scanning position (SCAN_POS) in the argument list. Starting at the SCAN_POS, *get_fld( )* reads characters from the buffer into a field until either a delimiter or the end of the record is reached. Function *get_fld( )* returns the SCAN_POS for use on the next call. Implementations of *writrec* and *readrec* in both C and Pascal are included along with the other programs at the end of this chapter.

### 4.1.6 Mixing Numbers and Characters: Use of a File Dump

File dumps give us the ability to look inside a file at the actual bytes that are stored there. Consider, for instance, the record-length information in the Pascal program output that we were examining a moment ago. The length of the Ames record, which is the first one in the file, is 40 characters, including delimiters. In the Pascal version of *writrec*, where we store the ASCII character representation of this decimal number, the actual bytes stored *in the file* look like the representation in Fig. 4.10(a). In the C implementation, where we choose to represent the length field as a two-byte integer, the bytes look like the representation in Fig. 4.10(b).

As you can see the *number* 40 is not the same as the set of characters '4' and '0'. The hex value of the *binary integer* 40 is 0x28; the hex values of the *characters* '4' and '0' are 0 x 34 and 0 x 30. (We are using the C language convention of identifying hexadecimal numbers through the use of the prefix 0x.) So, when we are storing a number in ASCII form, it is the hex

**FIGURE 4.10** The number 40, stored as ASCII characters and as a short integer.

| | Decimal value of number | Hex value stored in bytes | | ASCII character form |
|---|---|---|---|---|
| (a) 40 stored as ASCII chars: | 40 | 34 | 30 | '4' '0' |
| (b) 40 stored as a 2-byte integer: | 40 | 00 | 28 | '\0' '(' |

values of the *ASCII characters* that go into the file, not the hex value of the number itself.

Figure 4.10(b) shows the byte representation of the number 40 stored as an integer (this is called storing the number in *binary* form, even though we usually view the output as a hexadecimal number). Now the hexadecimal value stored in the file is that of the number itself. The ASCII characters that happen to be associated with the number's actual hexadecimal value have no obvious relationship to the number. Here is what the version of the file that uses binary integers for record lengths looks like if we simply print it on a terminal screen:

```
(Ames | John | 123 Maple | Stillwater | OK | 74075 | ☺Mason|Alan|…
```

0x28 is ascii code for '('

Blank, since '\0' is unprintable.

0x28 is ascii code for '('

Blank; '\0' is unprintable.

The ASCII representations of characters and numbers in the actual record come out nicely enough, but the binary representations of the length fields are displayed cryptically. Let's take a different look at the file, this time using the UNIX dump utility od. Entering the UNIX command

```
od -xc <filename>
```

produces the following:

```
Offset              Values
0000000 \0   (   A   m   e   s   |   J   o   h   n   |   1   2   3      ←ASCII
          0028  416d  6573  7c4a  6f68  6e7c  3132  3320              ←Hex
0000020 M   a   p   l   e   |   S   t   i   l   l   w   a   t   e   r
          4d61  706c  657c  5374  696c  6c77  6174  6572
0000040 |   O   K   |   7   4   0   7   5   |  \0   $   M   a   s   o
          7c4f  4b7c  3734  3037  357c  0024  4d61  736f
0000060 n   |   A   l   a   n   |   9   0       E   a   s   t   g   a
          6e7c  416c  616e  7c39  3020  4561  7374  6761
0000100 t   e   |   A   d   a   |   O   K   |   7   4   B   2   0   |
          7465  7c41  6461  7c4f  4b7c  3734  3832  307c
```

As you can see, the display is divided into three different kinds of data. The column on the left labeled *Offset* gives the offset of the first byte of the row that is being displayed. The byte offsets are given in octal form; since each line contains 16 (decimal) bytes, moving from one line to the next adds 020 to the range. Every pair of lines in the printout contains interpretations of the bytes in the file in hexadecimal and ASCII. These representations were requested on the command line with the *-xc* flag (*x* = "hex;" *c* = "character").

Let's look at the first row of ASCII values. As you would expect, the



data placed in the file in ASCII form appears in this row in a readable way. But there are hexadecimal values for which there is no printable ASCII representation. The only such value appearing in this file is 0 x 00. But there could be many others. For example, the hexadecimal value of the number 500,000,000 is 0x1DCD6500. If you write this value out to a file, an *od* of the file with the option *-xc* looks like this:

```
0000000 \035\315   e  \0
          1dcd 6500
```

The only printable byte in this file is the one with the value 0x65 ('e'). *Od* handles all of the others by listing their equivalent octal values in the ASCII representation.

The hex dump of this output from the C version of *writrec* shows how this file structure represents an interesting mix of a number of the organizational tools we have encountered. In a single record we have both binary and ASCII data. Each record consists of a fixed-length field (the byte count) and several delimited, variable-length fields. This kind of mixing of different data types and organizational methods is common in real-world file structures.

**A Note about Byte Order** If the computer you are using is an IBM PC or a computer from DEC, such as a VAX, your octal dump for this file will probably be different from the one we see here. These machines store the values of numbers in reverse order from the way we think of them. For example, if this dump were executed on an IBM PC, the hex representation of the first two-byte value in the file would be 0x2800, rather than 0x0028. This reverse order also applies to long, four-byte integers on these machines. This is an aspect of files that you need to be aware of if you expect to make sense out of dumps like this one. A more serious consequence of the byte-order differences among machines occurs when we move files from a machine with one type of byte ordering to one with a different byte ordering. We discuss this problem and ways to deal with it in section 4.6, "Portability and Standardization."

## 4.2 Record Access

### 4.2.1 Record Keys

Since our new file structure so clearly focuses on the notion of a record as the quantity of information that is being read or written, it makes sense to think in terms of retrieving just one specific record rather than having to read all the way through the file, displaying everything. When looking for

an individual record, it is convenient to identify the record with a *key* based on the record's contents. For example, in our name and address file we might want to access the "Ames record" or the "Mason record" rather than thinking in terms of the "first record" or "second record." (Can you remember which record comes first?) This notion of a *key* is another fundamental conceptual tool. We need to develop a more exact idea of what a key is.

When we are looking for a record containing the last name *Ames*, we want to recognize it even if the user enters the key in the form "AMES", "ames", or "Ames". To do this, we must define a standard form for keys, along with associated rules and procedures for converting keys into this standard form. A standard form of this kind is often called a *canonical form* for the key. One meaning of the word *canon* is rule, and the word *canonical* means conforming to the rule. A canonical form for a search key is the *single* representation for that key that conforms to the rule.

As a simple example, we could state that the canonical form for a key requires that the key consist solely of uppercase letters and have no extra blanks at the end. So, if a user enters "Ames", we would convert the key to the canonical form "AMES" before searching for it.

It is often desirable to have *distinct keys*, or keys that uniquely identify a single record. If there is not a one-to-one relationship between the key and a single record, then the program has to provide additional mechanisms to allow the user to resolve the confusion that can result when more than one record fits a particular key. Suppose, for example, that we are looking for John Ames's address. If there are several records in the file for several different people named John Ames, how should the program respond? Certainly it should not just give the address of the first John Ames that it finds. Should it give all the addresses at once? Should it provide a way of scrolling through the records?

The simplest solution is to *prevent* such confusion. The prevention takes place as new records are added to the file. When the user enters a new record, we form a unique canonical key for that record and then search the file for that key. This concern about uniqueness applies only to *primary keys*. A primary key is, by definition, the key that is used to identify a record uniquely.

It is also possible, as we see later, to search on *secondary keys*. An example of a secondary key might be the city field in our name and address file. If we wanted to find all the records in the file for people who live in towns named Stillwater, we would use some canonical form of "Stillwater" as a secondary key. Typically, secondary keys do not uniquely identify a record.

Although a person's name might at first seem to be a good choice for a primary key, a person's name runs a high risk of failing the test for



uniqueness. A name is a perfectly fine secondary key, and in fact is often an important secondary key in a retrieval system, but there is too great a likelihood that two names in the same file will be identical.

The reason a name is a risky choice for a primary key is that it contains a real data value. In general, *primary keys should be dataless*. Even when we think we are choosing a unique key, if it contains data there is a danger that unforeseen identical values could occur. Sweet (1985) cites an example of a file system that used a person's Social Security number as a primary key for personnel records. It turned out that, in the particular population that was represented in the file, a large number of people who were not U.S. citizens were included, and in a different part of the organization all of these people had been assigned the Social Security number 999–99–9999!

Another reason, other than uniqueness, that a primary key should be dataless is that a primary key should be *unchanging*. If information that corresponds to a certain record changes, and that information is contained in a primary key, what do you do about the primary key? You probably cannot change the primary key itself, in most cases, because there are likely to be reports, memos, indexes, or other sources of information that refer to the record by its primary key. As soon as you change the key, those references become useless.

A good rule of thumb is to avoid trying to put data into primary keys. If we want to access records according to data content, we should assign this content to secondary keys. We give a more detailed look at record access by primary and secondary keys in Chapter 6. For the rest of this chapter, we suspend our concern about whether a key is primary or secondary and concentrate simply on finding things by key.

### 4.2.2 A Sequential Search

Now that you know about keys, you should be able to write a program that reads through the file, record by record, looking for a record with a particular key. Such *sequential searching* is just a simple extension of our *readrec* program, adding a comparison operation to the main loop to see if the key for the record matches the key we are seeking. We leave the actual program as an exercise.

**Evaluating Performance of Sequential Search** In the chapters that follow, we find ways to search for records that are faster than the sequential search mechanism. We can use sequential searching as a kind of baseline against which to measure the improvements that we make. It is important, therefore, to find some way of expressing the amount of time and work expended in a sequential search.

Developing a performance measure requires that we decide on a unit of work that usefully represents the constraints on the performance of the whole process. When we describe the performance of searches that take place in electronic RAM, where comparison operations are more expensive than fetch operations to bring data in from memory, we usually use the *number of comparisons* required for the search as the measure of work. But, given that the cost of a comparison in RAM is so small compared to the cost of a disk access, comparisons do not fairly represent the performance constraints for a search through a file on secondary storage. Instead, we count low-level READ( ) calls. We assume that each READ( ) call requires a seek and that any one READ( ) call is as costly as any other. We know from the discussions of matters such as system buffering in Chapter 3 that these assumptions are not strictly accurate. But, in a multiuser environment where many processes are using the disk at once, they are close enough to correct to be useful.

Suppose we have a file with 1,000 records and we want to use a sequential search to find Al Smith's record. How many READ( ) calls are required? If Al Smith's record is the first one in the file, the program has to read in only a single record. If it is the last record in the file, the program makes 1,000 READ( ) calls before concluding the search. For an average search, 500 calls are needed.

If we double the number of records in a file, we also double both the average and the maximum number of READ( ) calls required. Using a sequential search to find Al Smith's record in a file of 2,000 records, requires, on the average, 1,000 calls. In other words, the amount of work required for a sequential search is directly proportional to the number of records in the file.

In general, the work required to search sequentially for a record in a file with $n$ records is proportional to $n$; it takes at most $n$ comparisons; on average it takes approximately $n/2$ comparisons. A sequential search is said to be of the order $O(n)$ because the time it takes is proportional to $n$.[†]

### Improving Sequential Search Performance with Record Blocking

It is interesting and useful to apply some of the information from Chapter 3 about disk performance to the problem of improving sequential search performance. We learned in Chapter 3 that the major cost associated with a disk access is the time required to perform a seek to the right location on the disk. Once data transfer begins, it is relatively fast, although still much slower than a data transfer within RAM. Consequently, the cost of seeking

[†]If you are not familiar with this "big-oh" notation, you should look it up. Knuth (1973a) is a good source.



and reading a record and then seeking and reading another record is greater than the cost of seeking just once and then reading two successive records. (Once again, we are assuming a multiuser environment in which a seek is required for each separate READ( ) call.) It follows that we should be able to improve the performance of sequential searching by reading in a *block* of several records all at once and then processing that block of records in RAM.

We began this chapter with a stream of bytes. We grouped the bytes into fields, and then grouped the fields into records. Now we are considering a yet higher level of organization, grouping records into blocks. This new level of grouping, however, differs from the others. Whereas fields and records are ways of maintaining the logical organization within the file, blocking is done strictly as a performance measure. As such, the block size is usually related more to the physical properties of the disk drive than to the content of the data. For instance, on sector-oriented disks the block size is almost always some multiple of the sector size.

Suppose we have a file of 4,000 records and that the average length of a record is 512 bytes. If our operating system uses sector-sized buffers of 512 bytes, then an unblocked sequential search requires, on the average, 2,000 READ( ) calls before it can retrieve a particular record. By blocking the records in groups of 16 per block, so each READ( ) call brings in 8 kilobytes worth of records, the number of reads required for an average search comes down to 125. Each READ( ) requires slightly more time, since more data is transferred from the disk, but this is a cost that is usually well worth paying for such a large reduction in the number of reads.

There are several things to note from this analysis and discussion of record blocking:

- Although blocking can result in substantial performance improvements, it does not change the order of the sequential search operation. The cost of searching is still $O(n)$, increasing in direct proportion to increases in the size of the file.
- Blocking clearly reflects the differences between RAM access speed and the cost of accessing secondary storage.
- Blocking does not change the number of comparisons that must be done in RAM, and it probably increases the amount of data transferred between disk and RAM. (We always read a whole block, even if the record we are seeking is the first one in the block.)
- Blocking saves time because it decreases the amount of seeking. We find, again and again, that this differential between the cost of seeking and the cost of other operations, such as data transfer or RAM access, is the force that drives file structure design.

**When Sequential Searching Is Good** Much of the remainder of this text is devoted to identifying better ways to access individual records; sequential searching is just too expensive for most serious retrieval situations. This is unfortunate, because sequential access has two major practical advantages over other types of access: It is extremely easy to program, and it requires the simplest of file structures.

Whether sequential search is advisable depends largely on how the file is to be used, how fast the computer system is that is performing the search, and structural aspects of the file. There are many situations in which a sequential search is often reasonable. Here are some examples:

- ASCII files in which you are searching for some pattern (see *grep* in the next section);
- Files with few records (e.g., 10 records);
- Files that hardly ever need to be searched (e.g., tape files usually used for other kinds of processing); and
- Files in which you want all records with a certain secondary key value, where a large number of matches is expected.

Fortunately, these sorts of applications do occur often in day-to-day computing—so often, in fact, that operating systems provide many utilities for performing sequential processing. UNIX is one of the best examples of this, as we see in the next section.

### 4.2.3 UNIX Tools for Sequential Processing

Recognizing the importance of having a standard file structure that is simple and easy to program, the most common file structure that occurs in UNIX is *an ASCII file with the new-line character as the record delimiter and, when possible, white space as the field delimiter.* Practically all files that we create using UNIX editors use this structure. And since most of the built-in C and Pascal functions that perform I/O write to this kind of file, it is common to see data files that consist of fields of numbers or words separated by blanks or tabs, and records separated by new-line characters. Such files are simple and easy to process. We can, for instance, generate an ASCII file with a simple program, and then use an editor to browse through it or alter it.

UNIX provides a rich array of tools for working with files in this form. Since this kind of file structure is inherently sequential (records are variable in length, so we have to pass from record to record to find any particular field or record), many of these tools process files sequentially.

Suppose, for instance, that we choose the white-space/new-line structure for our address file, ending every field with a tab and ending every record with a new line. While this causes some problems in distinguishing fields (a blank is white space, but it doesn't separate a field), and in that sense



is not an ideal structure, it buys us something very valuable: the full use of those UNIX tools that are built around the white-space/new-line structure. For example, we can print the file on our console using any of a number of utilities, such as

```
cat:

>cat myfile
Ames     John 123 Maple    Stillwater OK 74075
Mason    Alan 90 Eastgate Ada         OK 74820
```

Or we can use tools like wc and grep for processing the files.

**wc** The command wc ("word count") reads through an ASCII file sequentially and counts the number of lines (delimited by new lines), words (delimited by white space), and characters in a file:

```
>wc myfile
        2  14  76
```

**grep** It is common to want to find out if a text file has a certain word or character string in it. For ASCII files that can reasonably be searched sequentially, UNIX provides an excellent filter for doing this called *grep* (and its variants egrep and fgrep). The word *grep* stands for "generalized regular expression," which describes the type of pattern that grep is able to recognize. In its simplest form, grep searches sequentially through a file for a pattern, and then returns to standard output (the console) all the lines in the file that contain the pattern.

```
>grep Ada myfile
Mason    Alan    90 Eastgate Ada    OK    74820
```

We can also combine tools to create, on the fly, some very powerful file processing software. For example, to find the number of words in all records containing the word Ada:

```
>grep Ada | wc
          1   7   36
```

As we move through the text we will encounter a number of other powerful UNIX commands that sequentially process files with the basic white-space/new-line structure.

### 4.2.4 Direct Access

The most radical alternative to searching sequentially through a file for a record is a retrieval mechanism known as *direct access*. We have direct access to a record when we can seek directly to the beginning of the record and

read it in. Whereas sequential searching is an $O(n)$ operation, direct access is O(1); no matter how large the file is, we can still get to the record we want with a single seek.

Direct access is predicated on knowing where the beginning of the required record is. Sometimes this information about record location is carried in a separate index file. But, for the moment, we assume that we do not have an index. We assume, instead, that we know the *relative record number* (RRN) of the record that we want. The idea of an RRN is an important concept that emerges from viewing a file as a collection of records rather than a collection of bytes. If a file is a sequence of records, then the RRN of a record gives its position relative to the beginning of the file. The first record in a file has RRN 0, the next has RRN 1, and so forth.†

In our name and address file, we might tie a record to its RRN by assigning membership numbers that are related to the order in which we enter the records in the file. The person with the first record might have a membership number of 1001, the second a number of 1002, and so on. Given a membership number, we can subtract 1001 to get the RRN of the record.

What can we do with this RRN? Not much, given the file structures we have been using so far, which consist of variable-length records. The RRN tells us the relative position of the record we want in the sequence of records, but we still have to read sequentially through the file, counting records as we go, to get to the record we want. An exercise at the end of this chapter explores a method of moving through the file called *skip sequential* processing, which can improve performance somewhat, but looking for a particular RRN is still an $O(n)$ process.

To support direct access by RRN, we need to work with records of fixed, known length. If the records are all the same length, then we can use a record's RRN to calculate the *byte offset* of the start of the record relative to the start of the file. For instance, if we are interested in the record with an RRN of 546 and our file has a fixed-length record size of 128 bytes per record, we can calculate the byte offset as follows:

$$\text{Byte offset} = 546 \times 128 = 69{,}888.$$

In general, given a fixed-length record file where the record size is $r$, the byte offset of a record with an RRN of $n$ is

$$\text{Byte offset} = n \times r.$$

Programming languages and operating systems differ with regard to where this byte offset calculation is done and even with regard to whether

†In keeping with the conventions of C and Turbo Pascal, we assume that the RRN is a *zero-based* count. In some file systems, the count starts at 1 rather than 0.



byte offsets are used for addressing within files. In C (and the UNIX and MS-DOS operating systems), where a file is treated as just a sequence of bytes, the application program does the calculation and uses the *lseek( )* command to jump to the byte that begins the record. All movement within a file is in terms of bytes. This is a very low-level view of files; the responsibility for translating an RRN into a byte offset belongs wholly to the application program.

The PL/I language and the operating environments in which PL/I is often used (OS/MVS, VMS) are examples of a much different, higher-level view of files. The notion of a sequence of bytes is simply not present when you are working with record-oriented files in this environment. Instead, files are viewed as collections of records that are accessed by keys. The operating system takes care of the translation between a key and a record's location. In the simplest case, the key is, in fact, just the record's RRN, but the determination of actual location within the file is still not the programmer's concern.

If we limit ourselves to the use of standard Pascal, the question of seeking by bytes or seeking by records is not an issue: There is no seeking at all in standard Pascal. But, as we said earlier, many implementations of Pascal extend the standard definition of the language to allow direct access to different locations in a file. The nature of these extensions varies according to the differences in the host operating systems around which the extensions were developed. All the same, one feature that is consistent across implementations is that a file in Pascal always consists of elements of a single type. A file is a sequence of integers, characters, arrays, or records, and so on. Addressing is always in terms of this fundamental element size. For example, we might have a *file of datarec*, where *datarec* is defined as

```
TYPE datarec = packed array [0..64] of char;
```

Seeking within this file is in terms of multiples of the elementary unit *datarec*, which is to say in multiples of a 65-byte entity. If I ask to jump to *datarec* number 3 (zero-based count), I am jumping 195 bytes ($3 \times 65 = 195$) into the file.

## 4.3 More about Record Structures

### 4.3.1 Choosing a Record Structure and Record Length

Once we decide to fix the length of our records so we can use the RRN to give us direct access to a record, we have to decide on a record length. Clearly, this decision is related to the size of the fields we want to store in

the record. Sometimes the decision is easy. Suppose we are building a file of sales transactions that contain the following information about each transaction:

- A six-digit account number of the purchaser;
- Six digits for the date field;
- A five-character stock number for item purchased;
- A three-digit field for quantity; and
- A 10-position field for total cost.

These are all fixed-length fields; the sum of the field lengths is 30 bytes. Normally, we would simply stick with this record size, but if performance is so important that we need to squeeze every bit of speed out of our retrieval system, we might try to fit the record size to the block organization of our disk. For instance, if we intend to store the records on a typical sectored disk (see Chapter 3) with a sector size of 512 bytes or some other power of 2, we might decide to pad the record out to 32 bytes so we can place an integral number of records in a sector. That way, records will never span sectors.

The choice of a record length is more complicated when the lengths of the fields can vary, as in our name and address file. If we choose a record length that is the sum of our estimates of the largest possible values for all the fields, we can be reasonably sure that we have enough space for everything, but we also waste a lot of space. If, on the other hand, we are conservative in our use of space and fix the lengths of fields at smaller values, we may have to leave information out of a field. Fortunately, we can avoid this problem to some degree through appropriate design of the field structure *within* a record.

In our earlier discussion of record structures, we saw that there are two general approaches we can take toward organizing fields within a fixed-length record. The first, illustrated in Fig. 4.11(a), uses fixed-length fields inside the fixed-length record. This is the approach we took for the sales transaction file previously described. The second approach, illustrated in Fig. 4.11(b), uses the fixed-length record as a kind of standard-sized container for holding something that looks like a variable-length record.

The first approach has the virtue of simplicity: It is very easy to "break out" the fixed-length fields from within a fixed-length record. The second approach lets us take advantage of an averaging-out effect that usually occurs: The longest names are not likely to appear in the same record as the longest address field. By letting the field boundaries vary, we can make more efficient use of a fixed amount of space. Also, note that the two approaches are not mutually exclusive. Given a record that contains a number of truly fixed-length fields and some fields that have variable-length information, we might design a record structure that combines these two approaches.



| | | | | |
|---|---|---|---|---|
| Ames | John | 123 Maple | Stillwater | OK74075 |
| Mason | Alan | 90 Eastgate | Ada | OK74820 |

(a)

Ames|John|123 Maple|Stillwater|OK|74075| ← Unused space →

Mason|Alan|90 Eastgate|Ada|OK|74820| ← Unused space →

(b)

**FIGURE 4.11** Two fundamental approaches to field structure within a fixed-length record. (a) Fixed-length records with fixed-length fields. (b) Fixed-length records with variable-length fields.

The programs *update.c* and *update.pas*, which are included in the set of programs at the end of this chapter, use direct access to allow a user to retrieve a record, change it, and then write it back. These programs create a file structure that uses variable-length fields within fixed-length records. Given the variability in the length of the fields in our name and address file, this is an appropriate choice.

One of the interesting questions that must be resolved in the design of this kind of structure is that of distinguishing the real-data portion of the record from the unused-space portion. The range of possible solutions parallels that of the solutions for recognizing variable-length records in any other context: We can place a record-length count at the beginning of the record, we can use a special delimiter at the end of the record, we can count fields, and so on. Because both *update.c* and *update.pas* use a character string buffer to collect the fields, and because we are handling character strings differently in C than in Pascal (strings are null-terminated in C; we keep a byte count of the string length at the beginning of the Pascal strings), it is convenient to use a slightly different file structure for the two implementations. In the C version we fill out the unused portion of the record with null characters. In the Pascal version we actually place a fixed-length field (an integer) at the start of the record to tell how many of the bytes in the record are valid. As usual, there is no single right way to implement this file structure; instead we seek the solution that is most appropriate for our needs and situation.

Figure 4.12 shows the hex dump output from each of these programs. The output introduces a number of other ideas, such as the use of *header records*, which we discuss in the next section. For now, however, just look at the structure of the data records. We have italicized the length fields at the start of the records in the output from the Pascal program. Although we filled out the records created by the Pascal program with blanks to make the output more readable, this blank fill is unnecessary. The length field at the start of the record guarantees that we do not read past the end of the data in the record.

### 4.3.2 Header Records

It is often necessary or useful to keep track of some general information about a file to assist in future use of the file. A *header record* is often placed at the beginning of the file to hold this kind of information. For example, in some versions of Pascal there is no easy way to jump to the end of a file, even though the implementation supports direct access. One simple solution to this problem is to keep a count of the number of records in the file and to store that count somewhere. We might also find it useful to include information such as the length of the data records, the date and time of the file's most recent update, and so on. Header records can help make a file a self-describing object, freeing the software that accesses the file from having to know *a priori* everything about its structure, and hence making the file-access software able to deal with more variation in file structures.

The header record usually has a different structure than the data records in the file. The output from *update.c*, for instance, uses a 32-byte header record, whereas the data records each contain 64 bytes. Furthermore, the data records created by *update.c* contain only character data, whereas the header record contains an integer that tells how many data records are in the file.

Implementing a header record presents more of a challenge for the Pascal programmer. Recall that the Standard Pascal view of a file is one of a repeated collection of components, all of which are the same component type. Since a header record is fundamentally a different kind of record than the other records in a file, Pascal does not naturally support header records. In some cases, Pascal lets us get around this problem by using *variant records*. A variant record in Pascal is one that can have different meanings, depending on context. Unfortunately, a variant record cannot vary in size, so its use as a header record is constrained by the fact that it must be the same size as all other records in the file.

When faced with a language like Standard Pascal that strictly proscribes the types of records we can use in a file, we often find ourselves resorting

```
0000000 0002 0000 0000 0000 0000 0000 0000 0000  ................
0000020 0000 0000 0000 0000 0000 0000 0000 0000  ................

Header record
Record count in
first two bytes

0000040 416d 6573 7c4a 6f68 6e7c 3132 3320 4d61  Ames|John|123 Ma
0000060 706c 657c 5374 696c 6c77 6174 6572 7c4f  ple|Stillwater|O
0000100 4b7c 3734 3037 357c 0000 0000 0000 0000  K|74075|........
0000120 0000 0000 0000 0000 0000 0000 0000 0000  ................

First record

0000140 4d61 736f 6e7c 416c 616e 7c39 3020 4561  Mason|Alan|90 Ea
0000160 7374 6761 7465 7c41 6461 7c4f 4b7c 3734  stgate|Ada|OK|74
0000200 3832 307c 0000 0000 0000 0000 0000 0000  820|............
0000220 0000 0000 0000 0000 0000 0000 0000 0000  ................

Second record

(a)

0000000 0002 0000 0000 0000 0000 0000 0000 0000  ................
0000020 0000 0000 0000 0000 0000 0000 0000 0000  ................
0000040 0000 0000 0000 0000 0000 0000 0000 0000  ................
0000060 0000 0000 0000 0000 0000 0000 0000 0000  ................
0000100 0000 0000                                ..

Header record
Record count in
first two bytes

0000102       0028 416d 6573 7c4a 6f68 6e7c 3132  (.Ames|John|12
0000120 3320 4d61 706c 657c 5374 696c 6c77 6174  3 Maple|Stillwat
0000140 6572 7c4f 4b7c 3734 3037 357c 0020 2020  er|OK|74075|
0000160 2020 2020 2020 2020 2020 2020 2020 2020
0000200 2020 2020

First record
Integer in first
two bytes contains
the number of
bytes of data in
the record

0000204       0024 4d61 736f 6e7c 416c 616e  $.Mason|Alan
0000220 7c39 3020 4561 7374 6761 7465 7c41 6461  |90 Eastgate|Ada
0000240 7c4f 4b7c 3734 3832 307c 0020 2020 2020  |OK|74820|
0000260 2020 2020 2020 2020 2020 2020 2020 2020
0000300 2020 2020 2020 2020

Second record

(b)
```

**FIGURE 4.12** Two different record structures that carry variable-length fields in a fixed-length record. (a) Record structure created by *update.c*: fixed-length records containing variable-length fields that are terminated by a null character. (b) Record structure created by *update.pas*: fixed-length records beginning with a fixed-length (integer) field that indicates the number of usable bytes in the record's variable-length fields.

to tricks. We use such a trick in *update.pas:* We just use the initial integer field in the record for a different purpose in the header record. In the data records this field holds a count of the bytes of valid data within the record; in the header record it holds a count of the data records in the file.

Header records are a widely used, important file design tool. For example, when we reach the point where we are discussing the construction of tree-structured indexes for files, we see that header records are often placed at the beginning of the index to keep track of matters such as the RRN of the record that is the root of the index. We investigate some more elaborate uses of header records later in this chapter and also in subsequent chapters.

## 4.4 File Access and File Organization

In the course of our discussions in this chapter, we have looked at

- Variable-length records;
- Fixed-length records;
- Sequential access; and
- Direct access.

The first two of these relate to aspects of *file organization*. The second pair has to do with *file access*. The interaction between file organization and file access is a useful one; we need to look at it more closely before continuing with this chapter.

Most of what we have considered so far falls into the category of file organization:

- Can the file be divided into fields?
- Is there a higher level of organization to the file that combines the fields into records?
- Do all the records have the same number of bytes or fields?
- How do we distinguish one record from another?
- How do we organize the internal structure of a fixed-length record so we can distinguish between data and extra space?

We have seen that there are many possible answers to these questions and that the choice of a particular file organization depends on many things, including the file-handling facilities of the language you are using and the *use you want to make of the file.*

Using a file implies access. We looked first at sequential access, ultimately developing a *sequential search*. So long as we did not know where individual records began, sequential access was the only option open to us.



When we wanted *direct access*, we fixed the length of our records, and this allowed us to calculate precisely where each record began and to seek directly to it.

In other words, our desire for direct *access* caused us to choose a fixed-length record file *organization*. Does this mean that we can equate fixed-length records with direct access? Definitely not. There is nothing about our having fixed the length of the records in a file that precludes sequential access; we certainly could write a program that reads sequentially through a fixed-length record file.

Not only can we elect to read through the fixed-length records sequentially, but we can also provide direct access to *variable-length* records simply by keeping a list of the byte offsets from the start of the file for the placement of each record. We chose a fixed-length record structure in *update.c* and *update.pas* because it is simple and adequate for the data that we want to store. Although the lengths of our names and addresses vary, the variation is not so great that we cannot accommodate it in a fixed-length record.

Consider, however, the effects of using a fixed-length record organization to provide direct access to records that are documents ranging in length from a few hundred bytes to over a hundred kilobytes. Fixed-length records would be disastrously wasteful of space, so some form of variable-length record structure would have to be found. Developing file structures to handle such situations requires that you clearly distinguish between the matter of *access* and your options regarding *organization*.

The restrictions imposed by the language and file system used to develop your applications do impose limits on your ability to take advantage of this distinction between access method and organization. For example, the C language provides the programmer with the ability to implement direct access to variable-length records, since it allows access to any byte in the file. On the other hand, Pascal, even when seeking is supported, imposes limitations related to Pascal's definition of a file as a collection of elements that are all of the same *type* and, consequently, size. Since the elements must all be of the same size, direct access to variable-length records is difficult, at best, in Pascal.

## 4.5 Beyond Record Structures

Now that we have a grip on the concepts of organization and access, we look at some interesting new file organizations and more complex ways of accessing files. We want to extend the notion of a file beyond the simple idea of records and fields.

We begin with the idea of abstract data models. Our purpose here is to put some distance between the physical and the logical organization of files, to allow us to focus more on the information content of files and less on physical format.

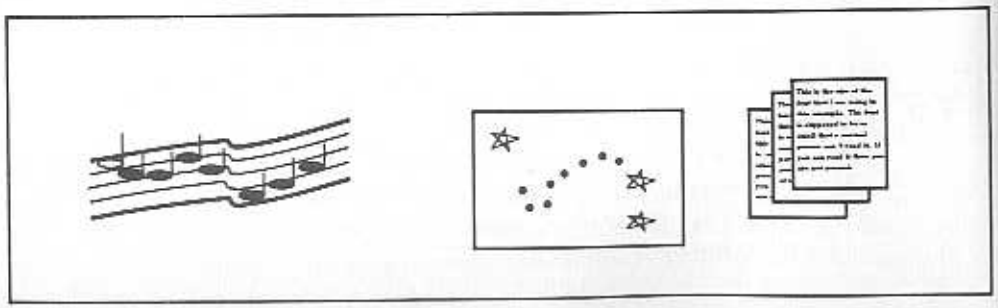
### 4.5.1 Abstract Data Models

The history of file structures and file processing parallels the history of computer hardware and software. When file processing first became common on computers, magnetic tape and punched cards were the primary means used to store files, RAM space was dear, and programming languages were primitive. Programmers as well as users were compelled to view file data exactly as it might appear on a tape or cards—as a sequence of fields and records. Even after data was loaded into RAM, the tools for manipulating and viewing the data were unsophisticated and reflected the magnetic tape metaphor. Data processing meant processing fields and records in the traditional sense.

Gradually, computer users began to recognize that computers could process more than just fields and records. Computers could, for instance, process and transmit sound, and they could process and display images and documents (Fig. 4.13). These kinds of applications deal with information that does not nicely fit the metaphor of data stored as sequences of records that are divided into fields, even if, ultimately, the data might be stored physically in the form of fields and records. It is easier, in the mind's eye, to envision data objects such as documents, images, and sound as objects that we manipulate in ways that are specific to the objects themselves, rather than simply as fields and records on a disk.

The notion that we need not view data only as it appears on a particular medium is captured in the phrase *abstract data model*, a term that encourages

**FIGURE 4.13** Data such as sound, images, and documents do not fit the traditional metaphor of data stored as sequences of records that are divided into fields.





an application-oriented view of data, rather than a medium-oriented view. The organization and access methods of abstract data models are described in terms of how an application views the data, rather than how the data might physically be stored.

One way that we save a user from having to know about objects in a file is to keep information in the file that file-access software can use to "understand" those objects. A good example of how this might be done is to put file structure information in a header.

### 4.5.2 Headers and Self-Describing Files

We have seen how a header record can be used to keep track of how many records there are in a file. If our programming language permits it, we can put much more elaborate information about a file's structure in the header. When a file's header contains this sort of information, we say the file is *self-describing*. Suppose, for instance, that we store in a file the following information:

- A name for each field;
- The width of each field; and
- The number of fields per record.

We can now write a program that can read and print a meaningful display of files with any number of fields per record and any variety of fixed-length field widths. In general, the more file structure information we put into a file's header, the less our software needs to know about the specific structure of an individual file.

As usual, there is a trade-off: If we do not hard-code the field and record structures of files in the programs that read and write them, the programs themselves must be more sophisticated. They must be flexible enough to interpret the self-descriptions that they find in the file headers.

### 4.5.3 Metadata

Suppose you are an astronomer interested in studying images generated by telescopes that scan the sky, and you want to design a file structure for the digital representations of these images (Fig. 4.14). You expect to have many images, perhaps thousands, that you want to study, and you want to store one image per file. While you are primarily interested in studying the images themselves, you will certainly need information *about* each image: where in the sky the image is from, when it was made, what telescope was used, references to related images, and so forth.

This kind of information is called *metadata*—data that describes the primary data in a file. Metadata can be incorporated into any file whose

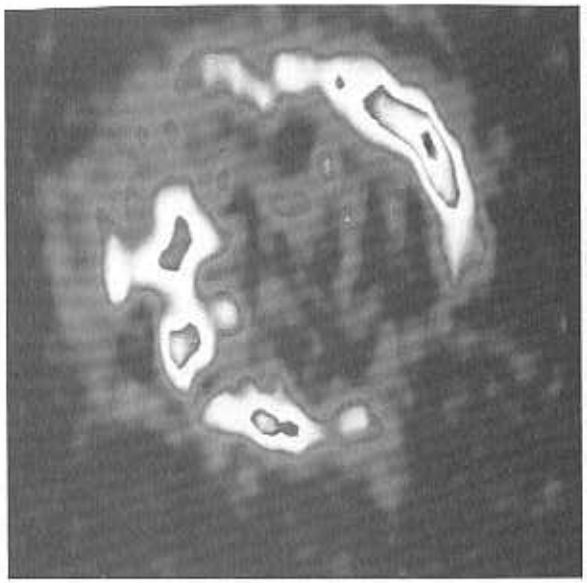


**FIGURE 4.14** To make sense of this two-Mbyte image, an astronomer needs such metadata as the kind of image it is, the part of the sky it is from, and the telescope that was used to view it. Astronomical metadata is often stored in the same file as the data itself. (This image shows polarized radio emission from the southern spiral galaxy NGC 5236 [M83] as observed with the Very Large Array radio telescope in New Mexico.)

primary data requires supporting information. If a file is going to be shared by many users, some of whom might not otherwise have easy access to its metadata, it may be most convenient to store the metadata in the file itself. A common place to store metadata in a file is the header record.

Typically, a community of users of a particular kind of data agrees on a standard format for holding metadata. For example, a standard format called FITS (Flexible Image Transport System) has been developed by the International Astronomers' Union for storing the kind of astronomical data just described in a file's header.[†] A FITS header is a collection of 2,880-byte blocks of 80-byte ASCII records, in which each record contains a single piece of metadata. Figure 4.15 shows part of a FITS header. In a FITS file, the header is followed by the actual numbers that describe the image, one binary number per observed point of the image.

Note that the designers of the FITS format chose to use ASCII in the header, but binary values for the image. ASCII headers are easy to read and process and, since they occur only once, take up relatively little extra space. Since the numbers that make a FITS image are rarely read by humans, but rather are first processed into a picture and then displayed, binary format is the preferred choice for them.

[†]For more details on FITS, see the references listed at the end of this chapter in "Further Readings."



```
SIMPLE  =                    T / CONFORMS TO BASIC FORMAT
BITPIX  =                   16 / BITS PER PIXEL
NAXIS   =                    2 / NUMBER OF AXES
NAXIS1  =                  256 / RA AXIS DIMENSION
NAXIS2  =                  256 / DEC AXIS DIMENSION
EXTEND  =                    F / T MEANS STANDARD EXTENSIONS EXIST
BSCALE  =    0.000100000 / TRUE = [TAPE*BSCALE]+BZERO
BZERO   =    0.000000000 / OFFSET TO TRUE PIXEL VALUES
MAP_TYPE= 'REL_EXPOSURE'/ INTENSITY OR RELATIVE EXPOSURE MAP
BUNIT   = '              '/ DIMENSIONLESS PEAK EXPOSURE FRACTION
CRVAL1  =          0.625 / RA   REF POINT VALUE (DEGREES)
CRPIX1  =        128.500 / RA   REF POINT PIXEL LOCATION
CDELT1  =  -0.006666700 / RA   INCREMENT ALONG AXIS (DEGREES)
CTYPE1  = 'RA---TAN'     / RA   TYPE
CROTA1  =          0.000 / RA   ROTATION
CRVAL2  =         71.967 / DEC  REF POINT VALUE (DEGREES)
CRPIX2  =        128.500 / DEC  REF POINT PIXEL LOCATION
CDELT2  =    0.006666700 / DEC  INCREMENT ALONG AXIS (DEGREES)
CTYPE2  = 'DEC--TAN'     / DEC  TYPE
CROTA2  =          0.000 / DEC  ROTATION
EPOCH   =         1950.0 / EPOCH OF COORDINATE SYSTEM
ARR_TYPE=              4 / 1=DP, 3=FP, 4=I
DATAMAX =          1.000 / PEAK INTENSITY (TRUE)
DATAMIN =          0.000 / MINIMUM INTENSITY (TRUE)
ROLL_ANG=        -22.450 / ROLL ANGLE (DEGREES)
BAD_ASP =              0 / 0=good, 1=bad(Do not use roll angle)
TIME_LIV=         5649.6 / LIVE TIME (SECONDS)
OBJECT  = 'REM6791       ' / SEQUENCE NUMBER
AVGOFFY =          1.899 / AVG Y OFFSET IN PIXELS, 8 ARCSEC/PIXEL
AVGOFFZ =          2.578 / AVG Z OFFSET IN PIXELS, 8 ARCSEC/PIXEL
RMSOFFY =          0.083 / ASPECT SOLN RMS Y PIXELS, 8 ARCSC/PIX
RMSOFFZ =          0.204 / ASPECT SOLN RMS Z PIXELS, 8 ARCSC/PIX
TELESCOP= 'EINSTEIN      ' / TELESCOPE
INSTRUME= 'IPC           ' / FOCAL PLANE DETECTOR
OBSERVER= '2             ' / OBSERVER #: 0=CFA; 1=CAL; 2=MIT; 3=GSFC
GALL    =        119.370 / GALACTIC LONGITUDE OF FIELD CENTER
GALB    =          9.690 / GALACTIC LATITUDE OF FIELD CENTER
DATE_OBS= '80/238        ' / YEAR & DAY NUMBER FOR OBSERVATION START
DATE_STP= '80/238        ' / YEAR & DAY NUMBER FOR OBSERVATION STOP
TITLE   = 'SNR SURVEY: CTA1                                        '
ORIGIN  = 'HARVARD-SMITHSONIAN CENTER FOR ASTROPHYSICS             '
DATE    = '22/09/1989 '  / DATE FILE WRITTEN
TIME    = '05:26:53   '  / TIME FILE WRITTEN
END
```

**FIGURE 4.15** Sample FITS header. On each line, the data to the left of the '/' is the actual metadata (data about the raw data that follows in the file). For example, the second line ("BITPIX = 16") indicates that the raw data in the file will be stored in 16-bit integer format. Everything to the right of a '/' is a comment, describing for the reader the meaning of the metadata that precedes it. Even a person uninformed about the FITS format can learn a great deal about this file just by reading through the header.

A FITS image is a good example of an abstract data model. The data itself is meaningless without the interpretive information contained in the header, and FITS-specific methods must be employed to convert FITS data into an understandable image. Another example is the raster image, which we look at next.

### 4.5.4 Color Raster Images

From a user's point of view, a modern computer is as much a graphical device as it is a data processor. Whether we are working with documents, spreadsheets, or numbers, we are likely to be viewing and storing pictures in addition to whatever other information we work with. Let's examine one type of image, the color raster image, as a means to filling in our conceptual understanding of data objects.

A color raster image is a rectangular array of colored dots, or *pixels*,† that are displayed on a screen. A FITS image is a raster image in the sense that the numbers that make up a FITS image can be converted to colors and then displayed on a screen. There are many different kinds of metadata that can go with a raster image, including

- The dimensions of the image: the number or pixels per row and the number of rows.
- The number of bits used to describe each pixel. This determines how many colors can be associated with each pixel. A 1-bit image can display only two colors, usually black and white. A 2-bit image can display four colors ($2^2$), an 8-bit image can display 256 colors ($2^8$), and so forth.
- A *color lookup table*, or *palette*, indicating which color is to be assigned to each pixel value in the image. A 2-bit image uses a color lookup table with 4 colors, an 8-bit image uses a table with 256 colors, and so forth.

If we think of an image as an abstract data type, what are some methods that we might associate with images? There are the usual ones associated with getting things in and out of a computer: a *read_image* routine and *store_image* routine. Then there are those that deal with images as special objects; for example,

- Display an image in a window on a console screen;
- Associate an image with a particular color lookup table;
- Overlay one image onto another to produce a composite image; and
- Display several images in succession, producing an animation.

†*Pixel* stands for "picture element."



The color raster image is an example of a type of data object that requires more than the traditional field/record file structure. This is particularly true when more than one image might be stored in a single file, or when we want to store a document or other complex object together with images in a file. Let's look at some ways to mix object types in one file.

### 4.5.5 Mixing Object Types in One File

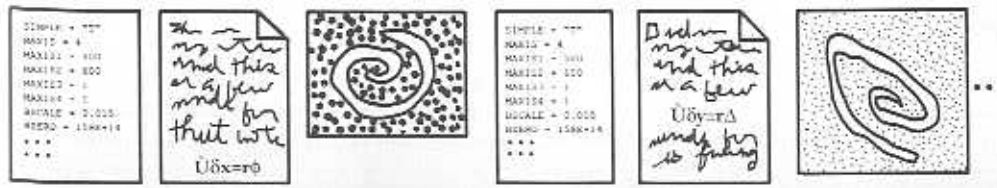
**Keywords** The FITS header (Fig. 4.15) illustrates an important technique, described earlier, for identifying fields and records: the use of *keywords*. In the case of FITS headers, we do not know what fields are going to be contained in any given header, so we identify each field using a "*keyword = value*" format.

Why does this format work for FITS files, whereas it was inappropriate for our address file? For the address file we saw that the use of keywords demanded a high price in terms of space, possibly even doubling the size of the file. In FITS files the amount of overhead introduced by keywords is quite small. When the image is included, the FITS file in the example contains approximately 2 megabytes. The keywords in the header occupy a total of about 400 bytes, or about 0.02% of the total file space.

**Tags** With the addition via keywords of file structure information and metadata to a header, we see that a file can be more than just a collection of repeated fields and records. Can we extend this notion beyond the header to other, more elaborate objects? For example, suppose an astronomer would like to store *several* FITS images of different sizes in a file, together with the usual metadata, plus perhaps lab notes describing what the scientist learned from the image (Fig. 4.16). Now we can think of our file as a mixture of objects that may be very different in content—a view that our previous file structures do not handle well. Maybe we need a new kind of file structure.

There are many ways to address this new file design problem. One would be simply to put each type of object into a variable-length record and

**FIGURE 4.16** Information that an astronomer wants to include in a file.



write our file processing programs so they know what each record looks like: The first record is a header for the first image; the second record is the image; the third record is a document; the fourth is a header for the second image; and so forth.

This solution is workable and simple, but it has some familiar drawbacks:

- Objects must be accessed sequentially, making access to individual images in large files time consuming.
- The file must contain exactly the objects that are described, in exactly the order indicated. We could not, for instance, leave out the notebook for some of the images (or in some cases leave out the notebook altogether) without rewriting all programs that access the file to reflect the changes in the file's structure.

A solution to these problems is hinted at in the FITS header: Each line begins with a keyword that identifies the metadata field that follows in the line. Why not use keywords to identify *all* objects in the file—not just the fields in the headers, but the headers themselves, as well as the images and any other objects we might need to store? Unfortunately, the "keyword = data" format makes sense in a FITS header—it is short and fits easily in an 80-byte line—but it doesn't work at all for objects that vary enormously in size and content. Fortunately, we can generalize the keyword idea to address these problems by making two changes:

- Lift the restriction that each record be 80 bytes, and let it be big enough to hold the object that is referenced by the keyword.
- Place the keywords in an index table, together with the byte offset of the actual metadata (or data) and a length indicator that indicates how many bytes the metadata (or data) occupies in the file.

The term *tag* is commonly used in place of *keyword* in connection with this type of file structure. The resulting structure is illustrated in Fig. 4.17. In it, we encounter two important conceptual tools for file design: (1) the use of an *index table* to hold descriptive information about the primary data, and (2) the use of *tags* to distinguish different types of objects. These tools allow us to store in one file a mixture of objects—objects that can vary from one another in structure and content.

Tag structures are common among standard file formats in use today. For example, a structure called TIFF (Tagged Image File Format) is a very popular tagged file format used for storing images. HDF (Hierarchical Data Format) is a standard tagged structure used for storing many different kinds of scientific data, including images. In the world of document storage and retrieval, SGML (Standard General Markup Language) is a *language* for



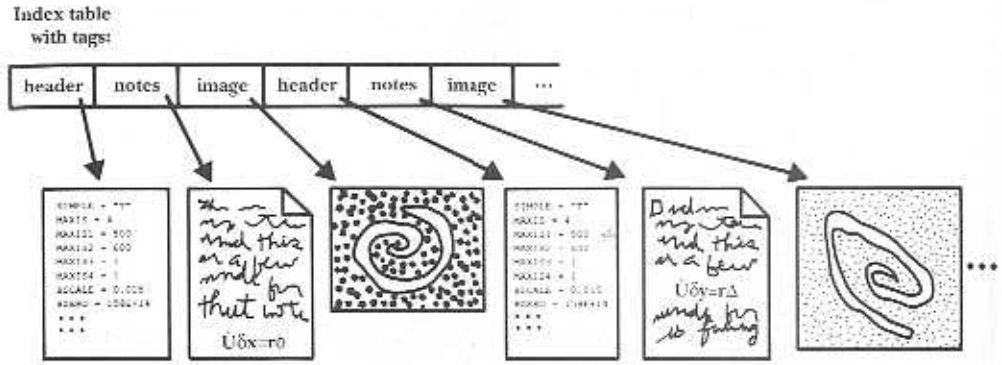



**FIGURE 4.17** Same as Fig. 4.16, except with tags identifying the objects.

describing document structures and for defining tags used to mark up that structure. Like FITS, each of these provides an interesting study in file design and standardization. References to further information on each are provided at the end of this chapter, in "Further Readings."

**Accessing Files with Mixtures of Data Objects** The idea of allowing files to contain widely varying objects is compelling, especially for applications that require large amounts of metadata or unpredictable mixes of different kinds of data, for it frees us of the requirement that all records be fundamentally the same. As usual, we must ask what this freedom costs us. To gain some insight into the costs, imagine that you want to write a program to access objects in such a file. You now have to read and write tags as well as data, and the structure and format for different data types are likely to be different. Here are some questions you will have to answer almost immediately:

- When we want to read an object of a particular type, how do we search for the object?
- When we want to store an object in the file, how and where do we store its tag, and where exactly do we put the object?
- Given that different objects will have very different appearances within a file, how do we determine the correct method for storing or retrieving the object?

The first two questions have to do with accessing the table that contains the tags and pointers to the objects. Solutions to this problem are dealt with in

detail in Chapter 6, so we defer their discussion until then. The third question, how to determine the correct methods for accessing objects, has implications that we briefly touch on here.

### 4.5.6 Object-oriented File Access

We have used the term *abstract data model* to describe the view that an application has of a data object. This is essentially an in-RAM, application-oriented view of an object, one that ignores the physical format of objects as they are stored in files. Taking this view of objects buys our software two things:

- It delegates to separate modules the responsibility of translating to and from the physical format of the object, letting the application modules concentrate on the task at hand. (For example, an image processing program that can operate in RAM on 8-bit images should not have to worry about the fact that a particular image comes from a file that uses the 32-bit FITS format.)
- It opens up the possibility of working with objects that at some level fit the same abstract data model, even though they are stored in different formats. The in-RAM representations of the images could be identical, even though they come from files with quite different formats.)

File access that exploits these possibilities could be called *object-oriented access*, emphasizing the parallels between it and the well-known object-oriented programming paradigm.

As an example that illustrates both points, suppose you have an image processing application program (we'll call it *find_star*) that operates in RAM on 8-bit images, and you need to process a collection of images. Some are stored in FITS files in a FITS format and some in TIFF files in a different format. An object-oriented approach (Fig. 4.18) would provide the application program with a routine (let's call it *read_image( )*) for reading images into RAM in the expected 8-bit form, letting the application concentrate on the image processing task. For its part, the routine *read_image( )*, given a file to get an image from, determines the format of the image within the file, invokes the proper procedure to read the image in that format, and converts it from that format into the 8-bit RAM format that the application needs.

Tagged file formats are one way to implement this conceptual view of file organization and file access. The specification of a tag can be accompanied by a specification of methods for reading, writing, and



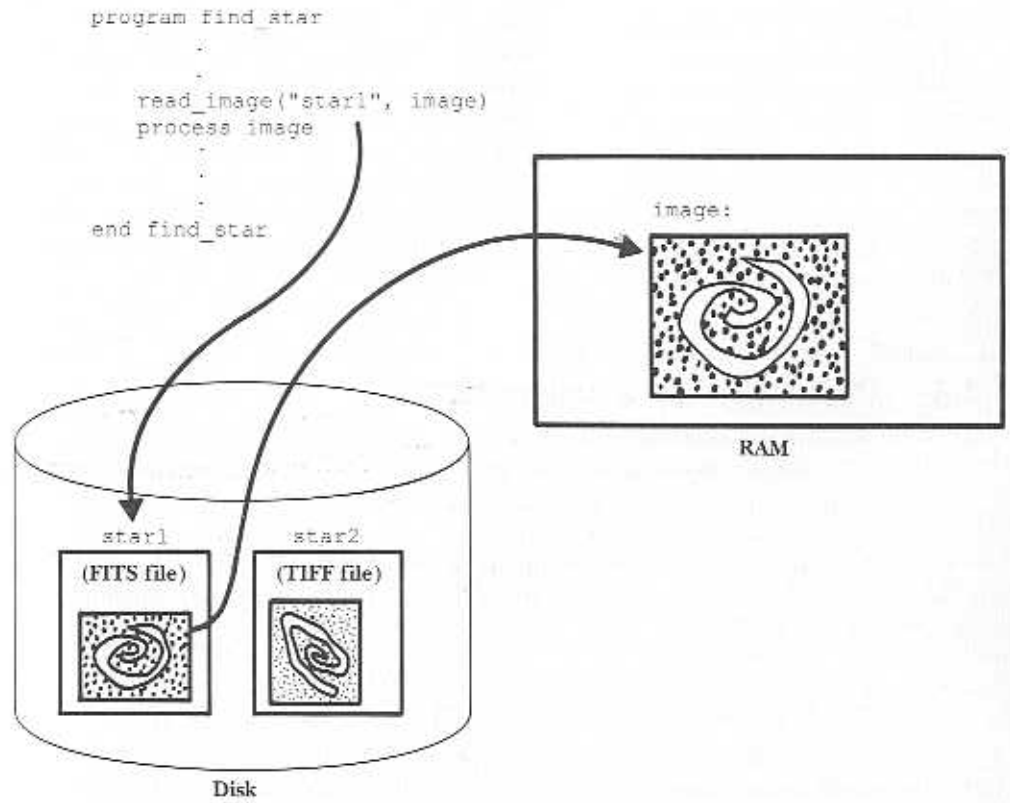



**FIGURE 4.18** Example of object-oriented access. The program *find_star* knows nothing about the file format of the image that it wants to read. The routine *read_image* has methods to convert the image from whatever format it is stored in on disk into the 8-bit in-RAM format required by *find_star*.

otherwise manipulating the corresponding data object according to the needs of an application. Indeed, any specification that separates the definition of the abstract data model from that of the corresponding file format lends itself to the object-oriented approach.

### 4.5.7 Extensibility

One of the advantages of using tags to identify objects within files is that we do not have to know *a priori* what all of the objects will look like that our software may eventually have to deal with. We have just seen that if our

program is to be able to access a mixture of objects in a file, it must have methods for reading and writing each object. Once we build into our software a mechanism for choosing the appropriate methods for a given type of object, it is easy to imagine extending, at some future time, the types of objects that our software can support. Every time we encounter a new type of object that we would like to accommodate in our files, we can implement methods for reading and writing that object and add those methods to the repertoire of methods available to our file processing software.

## 4.6 Portability and Standardization

A recurring theme in several of the examples that we have just seen is the idea that people often want to share files. Sharing files means making sure that they are accessible on all of the different computers that they might turn up on, and that they are somehow compatible with all of the different programs that will access them. In this final section, we look at two complementary topics that affect the sharability of files: portability and standardization.

### 4.6.1 Factors Affecting Portability

Imagine that you work for a company that wishes to share simple data files such as our address file with some other business. You get together with the other business to agree on a common field and record format, and you discover that your business does all of its programming and computing in C on a Sun computer and the other business uses Turbo Pascal on an IBM PC. What sorts of issues would you expect to arise?

**Differences among Operating Systems** In Chapter 2 in the section "Unexpected Characters in Files," we saw that MS-DOS adds an extra linefeed character every time it encounters a carriage return character, whereas on most other file systems this is not the case. This means that every time our address file has a byte with hex value 0x0d, whether or not that byte is meant to be a carriage return, the file is extended by an extra 0x0a byte.

This example illustrates the fact that *the ultimate physical format of the same logical file can vary depending on differences among operating systems.*

**Differences among Languages** Earlier in this chapter, when discussing header records, we chose to make our C header 32 bytes, but we were forced to make our Pascal header 64 bytes. C allows us to mix and match



fixed record lengths according to our needs, but Pascal requires that all records in a nontext file be the same size.

This illustrates a second factor impeding portability among files: *The physical layout of files produced with different languages may be constrained by the way the languages let you define structures within a file.*

**Differences in Machine Architectures** Consider again the header record that we produce in the C version of our address file. The hex dump of the file (Fig. 4.13), which was generated using C on a Sun 3 computer, shows this header record in the first line:

```
0000000  0020 0000 0000 0000 0000 0000 0000 0000
```

The first two bytes contain the number of records in the file, in this case $20_{16}$, or $32_{10}$. If the same C program is compiled and executed on an IBM PC or a VAX, the hex dump of the header record will look like this:

```
0000000  2000 0000 0000 0000 0000 0000 0000 0000
```

Why are the bytes reversed in this version of the program? The answer is that in both cases the numbers were written to the file exactly as they appeared in RAM, and the two different machines represent two-byte integers differently—the Sun stores the high-order byte, followed by the low-order byte; the IBM PC and VAX store the low-order byte, followed by the high-order byte.

This reverse order also applies to four-byte integers on these machines. For example, in our discussion of file dumps we saw that the hexadecimal value of $500{,}000{,}000_{10}$ is $1dcd6500_{16}$. If you write this value out to a file on an IBM PC, or some other reverse-order machine, a hex dump of the file created looks like this:

```
0000000  0065 cd1d
```

The problem of data representation is not restricted only to binary numbers. The way structures, such as C structs or Pascal records, are laid out in RAM can vary from machine to machine and compiler to compiler. For example, suppose you have a C program containing the following lines of code:

```
struct {
       int  cost;
       char ident[4];
} item
...
write (fd, &item, sizeof(item));
```

and you want to write files using this code on two different machines, a Cray 2 and a Sun 3. Because it likes to operate on 64-bit words, Cray's C

compiler allocates a minimum of eight bytes for any element in a struct, so it allocates 16 bytes for the struct *item*. When it executes the write( ) statement, then, the Cray writes 16 bytes to the file. The same program compiled on a Sun 3 writes only eight bytes, as you probably would expect, and on most IBM PCs it writes six bytes: same exact program; same language; three different results.

Text is also encoded differently on different platforms. In this case the differences are primarily restricted to two different types of systems: those that use EBCDIC[†] and those that use ASCII. EBCDIC is a standard created by IBM, so machines that need to maintain compatibility with IBM must support EBCDIC. Most others support ASCII. A few support both. Hence, text written to a file from an EBCDIC-based machine may well not be readable by an ASCII-based machine.

Equally serious, when we go beyond simple English text, is the problem of representing different character sets from different national languages. This is an enormous problem for developers of text databases.

### 4.6.2 Achieving Portability

Differences among languages, operating systems, and machine architectures represent three major problems when we need to generate portable files. Achieving portability means determining how to deal with these differences. And the differences are often not just differences between two platforms, for many different platforms could be involved.

The most important requirement for achieving portability is to recognize that it is not a trivial matter and to take steps ahead of time to insure it. Here are some guidelines.

**Agree on a Standard Physical Record Format and Stay with It** A physical standard is one that is represented the same physically, no matter what language, machine, or operating system is used. FITS is a good example of a physical standard, for it specifies exactly the physical format of each header record, the keywords that are allowed, the order in which keywords may appear, and the bit pattern that must be used to represent the binary numbers that describe the image.

Unfortunately, once a standard is established, it is very tempting to "improve" on it by changing it in some way, thereby rendering it no longer a standard. If the standard is sufficiently extensible, this temptation can sometimes be avoided. FITS, for example, has been extended a few times over its lifetime to support data objects that were not anticipated in its

[†]EBCDIC stands for Extended Binary Coded Decimal Interchange Code.



original design, yet all additions have remained compatible with the original format.

One way to make sure that a standard has staying power is to make it simple enough that files can be written in the standard format from a wide range of machines, languages, and operating systems. FITS again exemplifies such a standard. FITS headers are ASCII 80-byte records in blocks of 36 records each, and FITS images are stored as one contiguous block of numbers, both very simple structures that are easy to read and write in most modern operating systems and languages.

**Agree on a Standard Binary Encoding for Data Elements** The two most common types of basic data elements are text and numbers. In the case of text, ASCII and EBCDIC represent the most common encoding schemes, with ASCII standard on virtually all machines except IBM mainframes. Depending on the anticipated environment, one of these should be used to represent all text.[†]

The situation for binary numbers is a little cloudier. Although the number of different encoding schemes is not large, the likelihood of having to share data among machines that use different binary encodings can be quite high, especially when the same data is processed both on large mainframes and on smaller computers. Two standards efforts have helped diminish the problem, however: IEEE Standard formats, and External Data Representation (XDR).

IEEE has established standard format specifications for 32-bit, 64-bit, and 128-bit floating point numbers, and for 8-bit, 16-bit, and 32-bit integers. With a few notable exceptions (e.g., IBM mainframes, Cray, and Digital) most computer manufacturers have followed these guidelines in designing their machines. This effort goes a long way toward providing portable number encoding schemes.

XDR is an effort to go the rest of the way. XDR specifies not only a set of standard encodings for all files (the IEEE encodings), but provides for a set of routines for each machine for converting from its binary encoding when writing to a file, and vice versa (Fig. 4.19). Hence, when we want to store numbers in XDR, we can read or write them by replacing read and write routines in our program with XDR routines. The XDR routines take care of the conversions.[‡]

[†]Actually, there are different versions of both ASCII and EBCDIC. However, for most applications, and for the purposes of this text, it is sufficient to consider each as a single character set.

[‡]XDR is used for more than just number conversions. It allows a C programmer to describe arbitrary data structures in a machine-independent fashion. XDR originated as a Sun protocol for transmitting data that is accessed by more than one type of machine. For further information, see Sun (1986 or later).

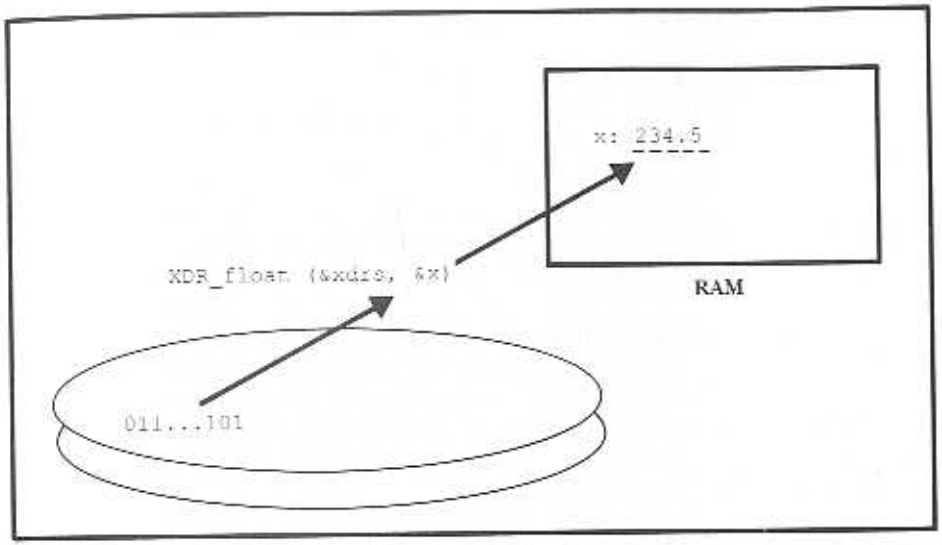



**FIGURE 4.19** XDR specifies a standard external data representation for numbers stored in a file. XDR routines are provided for converting to and from the XDR representation to the encoding scheme used on the host machine. Here a routine called `XDR_float( )` translates a 32-bit floating point number from its XDR representation on disk to that of the host machine.

Once again, FITS provides us with an excellent example: The binary numbers that constitute a FITS image must conform to the IEEE Standard. Any program written on a machine with XDR support can thus read and write portable FITS files.

**Number and Text Conversion** Sometimes the use of standard data encodings is not feasible. For example, suppose you are working primarily on IBM mainframes with software that deals with floating point numbers and text. If you choose to store your data in IEEE Standard formats, every time your program reads or writes a number or character it must translate the number from the IBM format to the corresponding IEEE format. This is not only time-consuming but can result in loss of accuracy. It is probably better in this case to store your data in native IBM format in your files.

What happens, then, when you want to move your files back and forth between your IBM and a VAX, which uses a different native format for numbers and generally uses ASCII for text? You need a way to convert from the IBM format to the VAX format and back. One solution is to write (or borrow) a program that translates IBM numbers and text to their VAX equivalents, and vice versa. This simple solution is illustrated in Fig. 4.20(a).



But what if, in addition to IBM and VAX computers, you find that your data is likely to be shared among many different platforms that use different numeric encodings? One way to solve this problem is to write a program to convert from each of the representations to every other representation. This solution, illustrated in Fig. 4.20(b), can get rather complicated. In general, if you have $n$ different encoding schemes, you will need $n(n-1)$ different translators. (Why?) If $n$ is large, this can be very messy. Not only do you need many translators, but you need to keep track, for each file, of where the file came from and/or where it is going in order to know which translator to use.

In this case, a better solution would probably be to agree on a standard intermediate format, such as XDR, and translate files into XDR whenever they are to be exported to a different platform. This solution is illustrated in Fig. 4.20(c). Not only does it cut down the number of translators from $n(n-1)$ to $2n$, but it should be easy to find translators to convert from most platforms to and from XDR. One negative aspect of this solution is that it requires *two* conversions to go from any one platform to another, a cost that has to be weighed against the complexity of providing $n(n-1)$ translators.

**File Structure Conversion** Suppose you are a doctor and you have X-ray raster images of a particular organ taken periodically over several minutes. You want to look at a certain image in the collection using a program that lets you zoom in and out and detect special features in the image. You have another program that lets you animate the collection of images, showing how it changes over several minutes. Finally, you want to annotate the images and store them in a special X-ray archive, and you have another program for doing that. What do you do if each of these three programs requires that your image be in a different format?

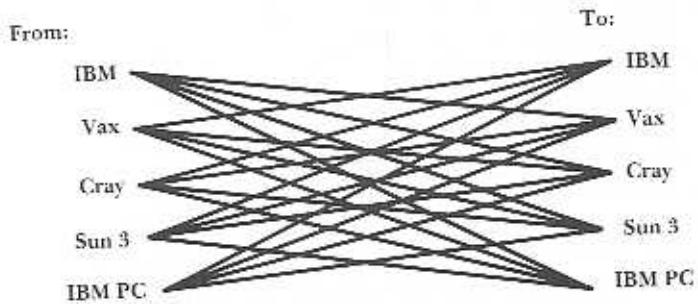
The conversion problems that apply to atomic data encodings also apply to file structures for more complex objects, like images, but at a different level. Whereas character and number encodings are tied closely to specific platforms, more complex objects and their representations just as often are tied to specific *applications*.

For example, there are many software packages that deal with images, and very little agreement about a file format for storing them. When we look at this software, we find different solutions to this problem:

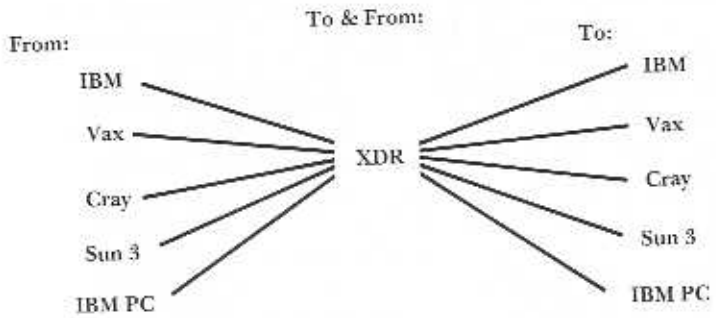
- Require that the user supply images in a format that is compatible with the one used by the package. This places the responsibility on the user to convert from one format to another. For such situations,




(a) Converting between IBM and Vax native format requires two conversion routines.




(b) Converting directly between five different native formats requires 20 conversion routines.




(c) Converting between five different native formats via an intermediate standard format requires 10 conversion routines.

**FIGURE 4.20** Direct conversion between $n$ native machines formats requires $n(n-1)$ conversion routines, as illustrated in (a) and (b). Conversion via an intermediate standard format requires $2n$ conversion routines, as illustrated in (c).



it may be preferable to provide utility programs that translate from one format to another and that are invoked whenever translating.

- Process only images that adhere to some predefined standard format. This places the responsibility on a community of users and software developers for agreeing on and enforcing a standard. FITS is a good example of this approach.
- Include different sets of I/O methods capable of converting an image from several different formats into a standard RAM structure that the package can work with. This places the burden on the software developer to develop I/O methods for file object types that may be stored differently but for the purposes of an application are conceptually the same. You may recognize this approach as a variation on the concept of *object-oriented access* that we discussed earlier.

**File System Differences** Finally, if you move files from one file system to another, chances are you will find differences in the way files are organized physically. For example, UNIX systems write files to tapes in 512-byte blocks, but non-UNIX systems often use different block sizes, such as 2,880-bytes—thirty-six 80-byte records. (Guess where the FITS blocking format comes from?) When transferring files between systems, you may need to deal with this problem.

**UNIX and Portability** Recognizing problems such as the block-size problem just described, UNIX provides a utility called *dd*. Although *dd* is intended primarily for copying tape data to and from UNIX systems, it can be used to convert data from any physical source. The *dd* utility provides the following options, among others:

- Convert from one block size to another;
- Convert fixed-length records to variable length, or vice versa;
- Convert ASCII to EBCDIC, or vice versa;
- Convert all characters to lowercase (or to uppercase); and
- Swap every pair of bytes.

Of course, the greatest contribution UNIX makes to the problems discussed here is UNIX itself. By its simplicity and ubiquity, UNIX encourages the use of the same operating system, the same file system, the same views of devices, and the same general views of file organization, no matter what particular hardware platform you happen to be using.

For example, one of the authors works in an organization with a nationwide constituency that operates many different computers, including two Crays, a Connection Machine, and many Sun, Apple, IBM, Silicon Graphics, and Digital workstations. Because each runs some flavor of

UNIX, they all incorporate precisely the same view of all external storage devices, they all use ASCII, and they all provide the same basic programming environment and file management utilities. Files are not perfectly portable within this environment, for reasons that we have covered in this chapter, but the availability of UNIX goes a long way toward facilitating the rapid and easy transfer of files among the applications, programming environments, and hardware systems that the organization supports.

## SUMMARY

The lowest level of organization that we normally impose on a file is a *stream of bytes*. Unfortunately, by storing data in a file merely as a stream of bytes, we lose the ability to distinguish among the fundamental informational units of our data. We call these fundamental pieces of information *fields*. Fields are grouped together to form *records*. Recognizing fields and recognizing records requires that we impose structure on the data in the file.

There are many ways to separate one field from the next and one record from the next:

- Fix the length of each field or record.
- Begin each field or record with a count of the number of bytes that it contains.
- Use delimiters to mark the divisions between entities.

In the case of fields, another useful technique is to use a "keyword = value" form to identify fields. In the case of records, another useful technique is to use a second, index file that tells where each record begins.

One higher level of organization, in which records are grouped into *blocks*, is also often imposed on files. This level is imposed to improve I/O performance rather than our logical view of the file.

In this chapter we use the record structure that uses a length indicator at the beginning of each record to develop programs for writing and reading a simple file of variable-length records containing names and addresses of individuals. We use buffering to accumulate the data in an individual record before we know its length to write it to the file. Buffers are also useful in allowing us to read in a complete record at one time. We represent the length field of each record as a binary number or as a sequence of ASCII digits. In the former case, it is useful to use a *file dump* to examine the contents of our file.



Sometimes we identify individual records by their *relative record numbers* (RRNs) in a file. It is also common, however, to identify a record by a *key* whose value is based on some of the record's content. Key values must occur in, or be converted to, some predetermined *canonical form* if they are to be recognized accurately and unambiguously by programs. If every record's key value is distinct from all others, the key can be used to identify and locate the unique record in the file. Keys that are used in this way are called *primary keys*.

In this chapter we look at the technique of searching sequentially through a file looking for a record with a particular key. Sequential search can perform poorly for long files, but there are times when sequential searching is reasonable. Record blocking can be used to improve the I/O time for a sequential search substantially. Two useful UNIX utilities that process files sequentially are *wc* and *grep*.

In our discussion of ways to separate records, it is clear that some of the methods provide a mechanism for looking up or calculating the *byte offset* of the beginning of a record. This, in turn, opens up the possibility of accessing the record *directly*, by RRN, rather than sequentially.

The simplest record formats for permitting direct access by RRN involve the use of fixed-length records. When the data itself actually comes in fixed-size quantities (e.g., zip codes), fixed-length records can provide good performance and good space utilization. If there is a lot of variation in the amount and size of data in records, however, the use of fixed-length records can result in expensive waste of space. In such cases the designer should look carefully at the possibility of using variable-length records.

Sometimes it is helpful to keep track of general information about files, such as the number of records they contain. A *header record*, stored at the beginning of the file it pertains to, is a useful tool for storing this kind of information.

It is important to be aware of the difference between *file access* and *file organization*. We try to organize files in such a way that they give us the types of access we need for a particular application. For example, one of the advantages of a fixed-length record *organization* is that it allows *access* that is either sequential or direct.

In addition to the traditional view of a file as a more or less regular collection of fields and records, we present a more purely logical view of the contents of files in terms of *abstract data models*, a view that lets applications ignore the physical structure of files altogether.

This view is often more appropriate to data objects such as sound, images, and documents. We call files *self-describing* when they do not require an application to reveal their structure, but provide that information

themselves. Another concept that deviates from the traditional view is *metadata*, in which the file contains data that describe the primary data in the file. FITS files, used for storing astronomical images, contain extensive headers with metadata.

The use of abstract data models, self-describing files, and metadata makes it possible to mix a variety of different types of data objects in one file. When this is the case, file access is more *object oriented*. Abstract data models also facilitate *extensible* files—files whose structures can be extended to accommodate new kinds of objects.

*Portability* becomes increasingly important as files are used in more heterogeneous computing environments. Differences among operating systems, languages, and machine architectures all lead to the need for portability. One important way to foster portability is *standardization*, which means agreeing on physical formats, encodings for data elements, and file structures.

If a standard does not exist and it becomes necessary to convert from one format to another, it is still often much simpler to have one standard format that all converters convert into and out of. UNIX provides a utility called *dd* that facilitates data conversion. The UNIX environment itself supports portability simply by being commonly available on a large number of platforms.

## KEY TERMS

**Block.** A collection of records stored as a physically contiguous unit on secondary storage. In this chapter, we use record blocking to improve I/O performance during sequential searching.

**Byte count field.** A field at the beginning of a variable-length record that gives the number of bytes used to store the record. The use of a byte count field allows a program to transmit (or skip over) a variable-length record without having to deal with the record's internal structure.

**Canonical form.** A standard form for a key that can be derived, by the application of well-defined rules, from the particular, nonstandard form of the data found in a record's key field(s) or provided in a search request supplied by a user.

**Delimiter.** One or more characters used to separate fields and records in a file.

**Direct access.** A file accessing mode that involves jumping to the exact location of a record. Direct access to a fixed-length record is usually accomplished by using its *relative record number* (RRN), computing its byte offset, and then seeking to the first byte of the record.



**Extensibility.** A characteristic of some file organizations that makes it possible to extend the types of objects that the format can accommodate without having to redesign the format. For example, tagged file formats lend themselves to extensibility, for they allow the addition of new tags for new data objects and associated new methods for accessing the objects.

**Field.** The smallest logically meaningful unit of information in a file. A record in a file is usually made up of several fields.

**File-access method.** The approach used to locate information in a file. In general, the two alternatives are *sequential access* and *direct access*.

**File organization method.** The combination of conceptual and physical structures used to distinguish one record from another and one field from another. An example of a kind of file organization is fixed-length records containing variable numbers of variable-length delimited fields.

**Fixed-length record.** A file organization in which all records have the same length. Records are padded with blanks, nulls, or other characters so they extend to the fixed length. Since all the records have the same length, it is possible to calculate the beginning position of any record, making *direct access* possible.

**Header record.** A record placed at the beginning of a file that is used to store information about the file contents and the file organization.

**Key.** An expression derived from one or more of the fields within a record that can be used to locate that record. The fields used to build the key are sometimes called the *key fields*. Keyed access provides a way of performing content-based retrieval of records, rather than retrieval based merely on a record's position.

**Metadata.** Data in a file that is not the primary data, but describes the primary data in a file. Metadata can be incorporated into any file whose primary data requires supporting information. If a file is going to be shared by many users, some of whom might not otherwise have easy access to its metadata, it may be most convenient to store the metadata in the file itself. A common place to store metadata in a file is the header record.

**Object-oriented file access.** A form of file access in which applications access data objects in terms of the applications' in-RAM view of the objects. Separate methods associated with the objects are responsible for translating to and from the physical format of the object, letting the application concentrate on the task at hand.

**Portability.** That characteristic of files that describes how amenable they are to access on a variety of different machines, via a variety of different operating systems, languages, and applications.

**Primary key.** A key that uniquely identifies each record and that is used as the primary method of accessing the records.

**Record.** A collection of related fields. For example, the name, address, etc. of an individual in a mailing list file would probably make up one record.

**Relative record number (RRN).** An index giving the position of a record relative to the beginning of its file. If a file has fixed-length records, the RRN can be used to calculate the *byte offset* of a record so the record can be accessed directly.

**Self-describing files.** Files that contain information such as the number of records in the file and formal descriptions of the file's record structure, which can be used by software in determining how to access the file. A file's header is a good place for this information.

**Sequential access.** Sequential access to a file means reading the file from the beginning and continuing until you have read in everything that you need. The alternative is direct access.

**Sequential search.** A method of searching a file by reading the file from the beginning and continuing until the desired record has been found.

**Stream of bytes.** Term describing the lowest-level view of a file. If we begin with the basic *stream-of-bytes* view of a file, we can then impose our own higher levels of order on the file, including field, record, and block structures.

**Variable-length record.** A file organization in which the records have no predetermined length. They are just as long as they need to be, hence making better use of space than fixed-length records do. Unfortunately, we cannot calculate the byte offset of a variable-length record by knowing only its relative record number.

## EXERCISES

1. Find situations for which each of the four field structures described in the text might be appropriate. Do the same for each of the record structures described.

2. Discuss the appropriateness of using the following characters to delimit fields or records: carriage return, linefeed, space, comma, period, colon, escape. Can you think of situations in which you might want to use different delimiters for different fields?

3. Suppose you want to change the programs in section 4.1 to include a phone number field in each record. What changes need to be made?

4. Suppose you need to keep a file in which every record has both fixed- and variable-length fields. For example, suppose you want to create a file of employee records, using fixed-length fields for each employee's ID (primary key), sex, birthdate, and department, and using variable-length fields for each name and address. What advantages might there be to using such a structure? Should we put the variable-length portion first or last? Either approach is possible; how can each be implemented?

5. One record structure not described in this chapter is called *labeled*. In a labeled record structure each field that is represented is preceded by a label describing its contents. For example, if the labels LN, FN, AD, CT, ST, and ZP are used to describe the six fixed-length fields for a name and address record, it might appear as follows:

```
LNAmesbbbbbbFNJohnbbbbbbAD123 MaplebbbbbbCTStillwaterSTOKZP74075bbbb
```

Under what conditions might this be a reasonable, even desirable, record structure?

6. Define the terms *stream of bytes*, *stream of fields*, and *stream of records*.

7. Find out what basic file structures are available to you in the programming language that you are currently using. For example, does your language recognize a sequence-of-bytes structure? Does it recognize lines of text? Record blocking? For those types of structures that your language does not recognize, describe how you might implement them using structures that your language does recognize.

8. Report on the basic field and record structures available in PL/I or COBOL.

9. Compare the use of ASCII characters to represent *everything* in a file with the use of binary and ASCII data mixed together.

10. If you list the contents of a file containing both binary and ASCII characters on your terminal screen, what results can you expect? What happens when you list a completely binary file on your screen? (*Warning:* If you actually try this, do so with a very small file. You could lock up or reconfigure your terminal, or even log yourself off!)

11. If a key in a record is already in canonical form and the key is the first

field of the record, it is possible to search for a record by key without ever separating out the key field from the rest of the fields. Explain.

12. It has been suggested (Sweet, 1985) that primary keys should be "dataless, unchanging, unambiguous, and unique." These concepts are interrelated since, for example, a key that contains data runs a greater risk of changing than a dataless key. Discuss the importance of each of these concepts, and show by example how their absence can cause problems. The primary key used in our example file violates at least one of the criteria. How might you redesign the file (and possibly its corresponding information content) so primary keys satisfy these criteria?

13. How many comparisons would be required on average to find a record using sequential search in a 10,000-record disk file? If the record is not in the file, how many comparisons are required? If the file is blocked so 20 records are stored per block, how many disk accesses are required on average? What if only one record is stored per block?

14. In our evaluation of performance for sequential search, we assume that every read results in a seek. How do the assumptions change on a single-user machine with access to a magnetic disk? How do these changed assumptions affect the analysis of sequential searching?

15. Look up the differences between the UNIX commands grep, egrep, and fgrep. Why are they different? What motivates the differences?

16. Give a formula for finding the byte offset of a fixed-length record in which the RRN of the first record is 1 rather than 0.

17. Why is a variable-length record structure unworkable for the *update* program? Does it help if we have an index that points to the beginning of each variable-length record?

18. The *update* program lets the user change records, but not delete records. How must the file structure and access procedures be modified to allow for deletion if we do not care about reusing the space from deleted records? How do the file structures and procedures change if we do want to reuse the space?

19. In our discussion of the uses of relative record numbers (RRNs), we suggest that you can create a file in which there is a direct correspondence between a primary key, such as membership number, and RRN, so we can find a person's record by knowing just the name or membership number. What kinds of difficulties can you envision with this simple correspondence between membership number and RRN? What happens if we want to delete



a name? What happens if we change the information in a record in a variable-length record file and the new record is longer?

20. The following file dump describes the first few bytes from a file of the type produced by the C version of *writrec*, but the right-hand column is not filled in. How long is the first record? What are its contents?

```
0000000 00264475 6D707C46 7265647C 38323120 ................
0000020 4B6C7567 657C4861 636B6572 7C50417C ................
0000040 36353533 357C2E2E 48657861 64656369 ................
```

21. Assume that we have a variable-length record file with long records (greater than 1,000 bytes each, on the average). Assume that we are looking for a record with a particular RRN. Describe the benefits of using the contents of a byte count field to skip sequentially from record to record to find the one we want. This is called *skip sequential* processing. Use your knowledge of system buffering to describe why this is useful only for long records. If the records are sorted in order by key and blocked, what information do you have to place at the start of each block to permit even faster skip sequential processing?

22. Suppose you have a fixed-length record with fixed-length fields, and the sum of the field lengths is 30 bytes. A record with a length of 30 bytes would hold them all. If we intend to store the records on a sectored disk with 512-byte sectors (see Chapter 3), we might decide to pad the record out to 32 bytes so we can place an integral number of records in a sector. Why would we want to do this?

23. Why is it important to distinguish between file access and file organization?

24. What is an abstract data model? Why did the early file processing programs not deal with abstract data models? What are the advantages of using abstract data models in applications? In what way does the UNIX concept of standard input and standard output conform to the notion of an abstract data model? (See "Physical and Logical Files in UNIX" in Chapter 2.)

25. What is metadata?

26. In the FITS header in Fig. 4.15, some metadata provides information about the files's structure, and some provides information about the scientific context in which the corresponding image was recorded. Give three examples of each.

27. In the FITS header in Fig. 4.15, there is enough information for a program to determine how to read the entire file. Assuming that the size of the block containing the header must be a multiple of 2,880 bytes, how large is the file? What proportion of the file contains header information?

28. In the discussion of field organization, we list the "keyword = value" construct as one possible type of field organization. How is this notion applied in tagged file structures? How does a tagged file structure support object-oriented file access? How do tagged file formats support extensibility?

29. List three factors that affect portability in files.

30. List three ways that portability can be achieved in files.

31. What is XDR? XDR is actually much more extensive than what we described in this chapter. If you have access to XDR documentation (see "Further Readings" at the end of this chapter), look up XDR and list the ways that it supports portability.

32. In Fig. 4.2, we see two possible record structures for our address file, one based on C and one based on Pascal. Discuss portability problems that might arise from using these record structures in a heterogeneous computing environment. (*Hint:* Some compilers allocate space for character fields starting on word boundaries, and others do not.)

## Programming Exercises

33. Rewrite *writstrm* so it uses delimiters as field separators. The output of the new version of *writstrm* should be readable by *readstrm.c* or *readstrm.pas*.

34. Create versions of *writrec* and *readrec* that use the following fixed-field lengths rather than delimiters.

| | |
|---|---|
| Last name: | 15 characters |
| First name: | 15 characters |
| Address: | 30 characters |
| City: | 20 characters |
| State: | 2 characters |
| Zip: | 5 characters |

35. Write the program described in the preceding problem so it uses blocks. Make it store five records per block.

36. Implement the program *find*.



37. Rewrite the program *find* so it can find a record on the basis of its position in the file. For example, if requested to find the 547th record in a file, it would read through the first 546 records, then print the contents of the 547th record. Use skip sequential search (see exercise 21) to avoid reading the contents of unwanted records.

38. Write a program similar to *find*, but with the following differences. Instead of getting record keys from the keyboard, the program reads them from a separate transaction file that contains only the keys of the records to be extracted. Instead of printing the records on the screen, it writes them out to a separate output file. First, assume that the records are in no particular order. Then assume that both the main file and the transaction file are sorted by key. In the latter case, how can you make your program more efficient than *find*?

39. Make any or all of the following alterations to *update.pas* or *update.c*.
    a. Let the user identify the record to be changed by name, rather than RRN.
    b. Let the user change individual fields without having to change an entire record.
    c. Let the user choose to view the entire file.

40. Modify *update.c* or *update.pas* to signal the user when a record exceeds the fixed-record length. The modification should allow the user to bring the record down to an acceptable size and input it again. What are some other modifications that would make the program more robust?

41. Change *update.c* or *update.pas* to a batch program that reads a transaction file in which each transaction record contains an RRN of a record that is to be updated, followed by the new contents of the record, and then makes the changes in a batch run. Although not necessary, it might be desirable to sort the transaction file by RRN. Why?

42. Write a program that reads a file and outputs the file contents as a file dump. The file dump should have a format similar to the one used in the examples in this chapter. The program should accept the name of the input file on the command line. Output should be to standard output (terminal screen).

43. Develop a set of rules for translating the dates August 7, 1949, Aug. 7, 1949, 8-7-49, 08-07-49, 8/7/49, and other, similar variations into a common canonical form. Write a function that accepts a string containing a date in one of these forms and returns the canonical form, according to your rules. Be sure to document the limitations of your rules and function.

44. Write a program to read in a FITS file and print
    a. The size of the image (e.g., 256 by 256)
    b. The title of the image
    c. The telescope used to make the image
    d. The date the image file was created
    e. The average pixel value in the image (use BSCALE and BZERO).

## FURTHER READINGS

Many textbooks cover basic material on field and record structure design, but only a few go into the options and design considerations in much detail. Teorey and Fry (1982) and Wiederhold (1983) are two possible sources. Hanson's (1982) chapter, "Choice of File Organization," is excellent but is more meaningful after you read the material in the later chapters of this text. You can learn a lot about alternative types of file organization and access by studying descriptions of options available in certain languages and file management systems. PL/1 offers a particularly rich set of alternatives, and Pollack and Sterling (1980) describe them thoroughly.

Sweet (1985) is a short but stimulating article on key field design. A number of interesting algorithms for improving performance in sequential searches are described in Gonnet (1984) and, of course, Knuth (1973b). Lapin (1987) provides a detailed coverage of portability in UNIX and C programming. For our coverage of XDR, we used the documentation in Sun (1986).

Our primary source of information on FITS is not formally printed text, but online materials. A good paper defining the original FITS format is Wells (1981). The FITS image and FITS header shown in this chapter, as well as the documentation of how FITS works, can (at the time of writing, at least) be found on an anonymous ftp server at the INTERNET address 128.183.10.4.



# C Programs

The C programs listed in the following pages correspond to the programs discussed in the text. The programs are contained in the following files.

| | |
|---|---|
| *writstrm.c* | Writes out name and address information as a stream of consecutive bytes. |
| *readstrm.c* | Reads a stream file as input and prints it to the screen. |
| *writrec.c* | Writes a variable-length record file that uses a byte count at the beginning of each record to give its length. |
| *readrec.c* | Reads through a file, record by record, displaying the fields from each of the records on the screen. |
| *getrf.c* | Contains support functions for reading individual records or fields. These functions are needed by programs in *readrec.c* and *find.c* |
| *find.c* | Searches sequentially through a file for a record with a particular key. |
| *makekey.c* | Combines first and last names and converts them to a key in canonical form. Calls *strtrim( )* and *ucase( )*, found in *strfuncs.c*. |
| *strfuncs.c* | Contains two string support functions: *strtrim( )* trims the blanks from the ends of strings; *ucase( )* converts alphabetic characters to uppercase. |
| *update.c* | Allows new records to be added to a file or old records to be changed. |

## Fileio.h

All of the programs include a header file called *fileio.h* which contains some useful definitions. Some of these are system dependent. If the programs were to be run on a UNIX system *fileio.h* might look like this:

```
/*  fileio.h --- header file containing file I/O definitions
*/
```

(continued)